SERVICIO DE VIDEO STREAMING

Cristóbal Suárez Abad

SERVICIOS DE RED E INTERNET - 2º ASIR

Índice

AWS 4
Configurar Instancia EC2 4
Instalar Docker y Docker Compose 8
Cambiar el hostname: 8
El archivo /etc/hosts 8
Primer Bloque: Jellyfin, Owncast y recorder 9
Jellyfin 15
Gestión masiva de usuarios: 17
Añadir retransmisión en directo: 20
Ver en otros dispositivos: 23
Owncast 27
recorder 35
Segundo Bloque: Jitsi Meet 37
Proceso de instalación: 37
Configurar archivos 39
Configuración de .env 40
Gestión de usuarios: 44
Levantar contenedores: 45
Usando Jitsi 46
Funciones 51
Seguridad: 56
Grabación de Videoconferencias: 59
Tercer bloque: Nginx Proxy Manager 63
Configuración de usuario Becario. Gestión de contenido multimedia. 68
Scripts para la gestión de contenedores. 73

Tomando como orientación el esquema de laboratorio virtual propuesto, implementar uno o varios servicios de streaming de video y de videoconferencia para el dominio ***2asirXX.institutodh.net*** *que trate de resolver las siguientes cuestiones:*

1. *Queremos crear un* ***repositorio multimedia de audio y vídeo*** *con contenido educativo para acceso de alumnos y profesores. Sería muy interesante la posibilidad de que los usuarios pudieran sindicarse para estar al tanto de las novedades que se vayan incorporando. Sería tb. Interesante poder acceder al contenido desde una Smart TV.*
2. *Se quiere establecer la configuración necesaria para realizar una* ***retransmisión en tiempo real*** *de las principales partidas del próximo torneo de Ajedrez interescolar. También se tiene en mente retransmitir en vivo el torneo deportivo de fútbol sala del día de la primavera. En esta línea, se quiere crear un taller de introducción al streaming para un grupo de alumnos interesados.*
3. *De otra parte, se quieren implementar un* ***servicio de videoconferencia****, especialmente orientado a la actividad académica*

PROCEDIMIENTO

Realizar:

1. *Realiza un análisis previo del software servidor elegido (plataformas, última versión, libre/propietaria, funcionalidad, clientes soportados, ...),*
2. *Instalación y configuración del servidor asignado para abordar uno o varios de los problemas planteados.*
3. *Implementad una prueba simulada, analizando las posibilidades de conexión de los clientes, costes derivados, etc...*
4. *Se valorará el uso de* ***DNS****,* ***docker*** *y* ***AWS*** *para el despliegue del servicio de streaming.*

Listado de servidores de streaming de video propuestos:

- *Opción 1.- Servidor **Jellyfin***
- *Opción 2.- Servidor **PeerTube***
- *Opción 3.- Servidor **Stash***
- *Opción 4.- **Universal Media Server***
- *Opción 5.- Servidor streaming multimedia **Streama***
- *Opción 6.- Nextcloud*
- *Opción 7.- Servidor Immich*
- *Opción 8. - Servicio de Videoconferencia con **True Conf**.*
- *Opción 8.5.- Kodi*
- *Opción 9.- Plex (propietario)*
- *Opción 10.- Streaming de audio/vídeo con **VLC** (streaming estándar y batch (unicast/difusión))*

Fuentes de documentación recomendadas:

1. *La web del Instituto Abierto de Cataluña: Actividades streaming video*
2. *La web oficial del desarrollador de cada servidor.*
    a. *TrueConf*
    b. *Streama, instala tu propio servidor de Streaming en Ubuntu | Ubunlog*
3. *Videotutoriales:*
    a. *• Streaming multiplataforma desde 0: Cómo hacer STREAM en Twitch, Mixer, YouTube, Facebook || Desde cero 🎮(2020)*
    b. *• Directos en Twitch:*
        i. *🔔 COMO hacer STREAM en TWITCH 🔔 Como Transmitir en Twitch !!*
        ii. *How to STREAM on TWITCH! *5 MIN* (2022 Tutorial) (PC)*
    c. *• Introducción a las emisiones en directo: Introducción a las emisiones en directo - Android - Ayuda de YouTube*

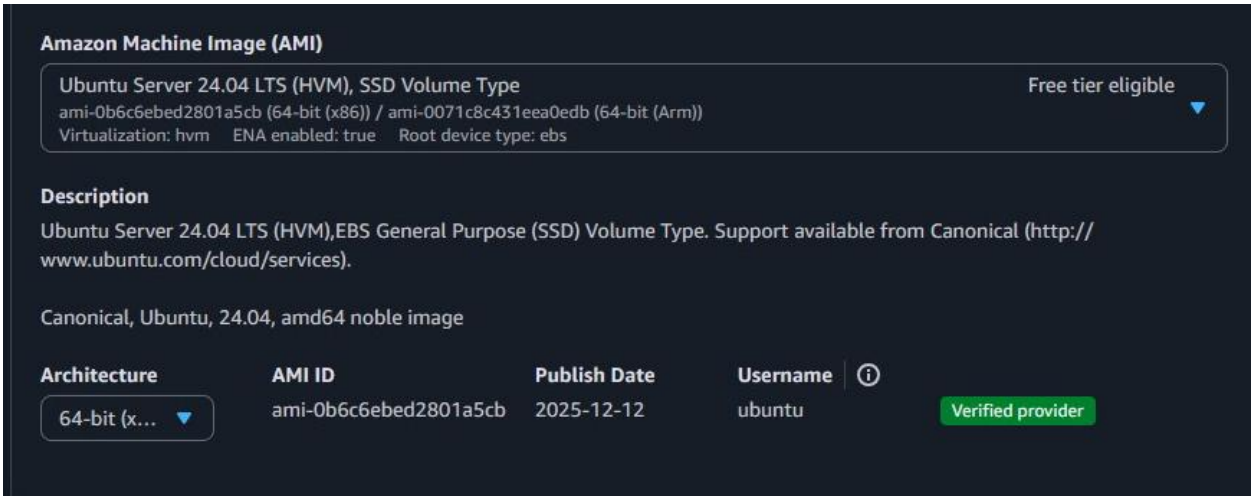
AWS

Configurar Instancia EC2

EC2 → Instances → Lanzar Instancias:

Elegimos Ubuntu, versión LTS de 64 bits.



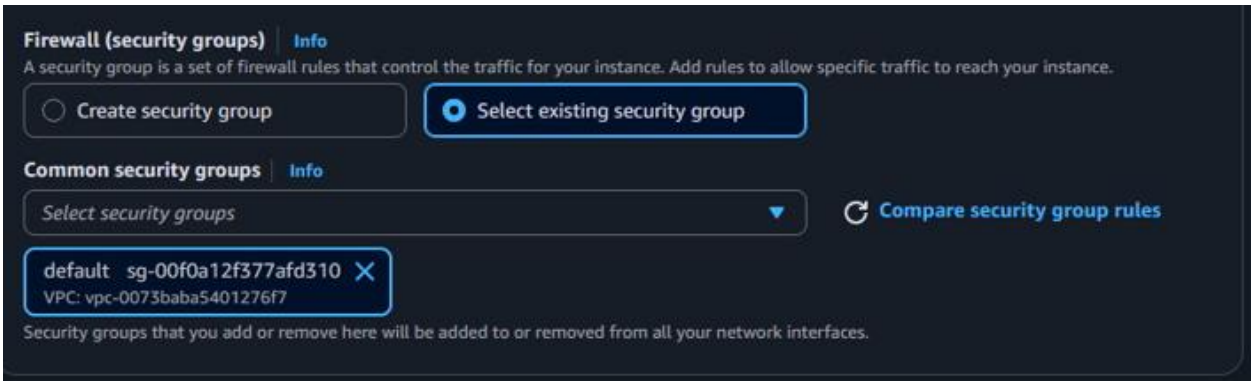


Tipo de Instancia:

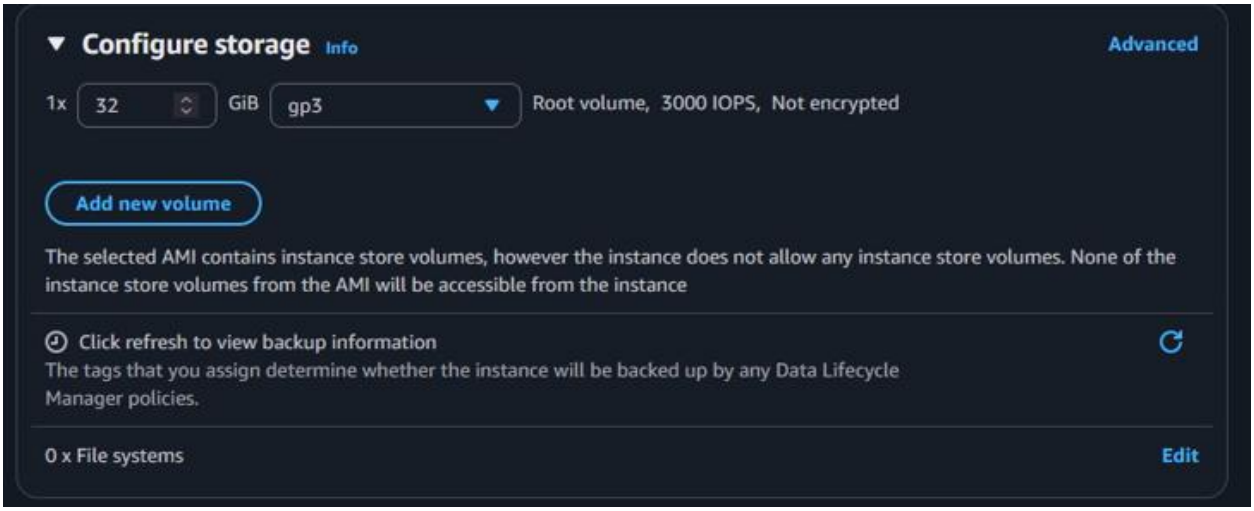
En esta ocasión vamos a usar una bastante grande.



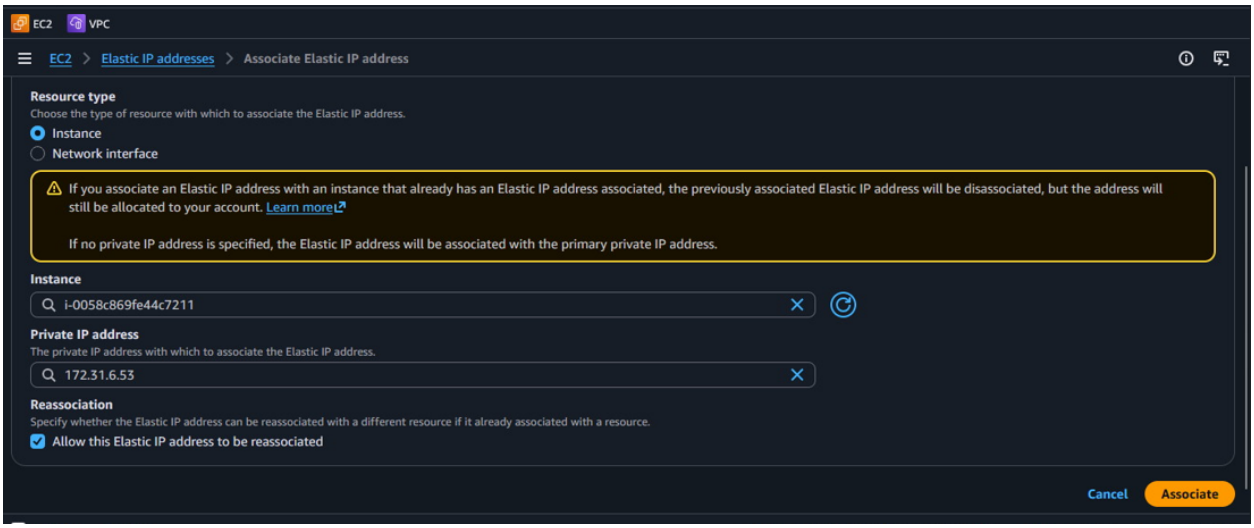
Firewall:



Almacenamiento:



Le asignamos una IP Elástica: 100.48.114.98



NOTA: Esta parte de la configuración se hace mediante avancemos con la instalación de contenedores. Muy importante fijarse en los puertos indicados en los archivos ".yml".

Type	Protocol	Port ra...	Source	Description
SSH	TCP	22	0.0.0.0/0	SSH
Custom TCP	TCP	8081	0.0.0.0/0	Owncast - Web
Custom TCP	TCP	1935	0.0.0.0/0	Owncast - Income Signal
Custom TCP	TCP	81	0.0.0.0/0	Nginx Proxy Manager - 81
HTTP	TCP	80	0.0.0.0/0	Nginx Proxy Manager - 80
HTTPS	TCP	443	0.0.0.0/0	Nginx Proxy Manager - ...
Custom UDP	UDP	10000	0.0.0.0/0	jvb
Custom TCP	TCP	8080	0.0.0.0/0	jvb
Custom TCP	TCP	8443	0.0.0.0/0	jitsi - Prueba - Luego us...
Custom TCP	TCP	8888	0.0.0.0/0	Jicofo
Custom TCP	TCP	8096	0.0.0.0/0	Jellyfin
All ICMP - IPv4	ICMP	All	0.0.0.0/0	-

Una vez iniciada la instancia, accedemos a ella vía **SSH**. Para la autenticación, utilizamos el archivo de clave privada "**.pem**" proporcionado por AWS. Dado que hemos asociado una **IP Elástica** que ya apunta a nuestro dominio de prácticas, podemos conectar indistintamente mediante el FQDN o la dirección IP pública:

ssh -i labuser.pem ubuntu@im07.institutodh.net

o también:

ssh -i labuser.pem ubuntu@100.48.114.98

Instalar Docker y Docker Compose

Lo primero que vamos a hacer es actualizar los repositorios e instalar Docker y Docker Compose.

apt update && apt install docker.io -y && apt install docker-compose-v2 -y

Cambiar el hostname:

Establecemos el nombre de dominio completo (FQDN) en el sistema para que los servicios generen correctamente sus identidades internas y certificados.

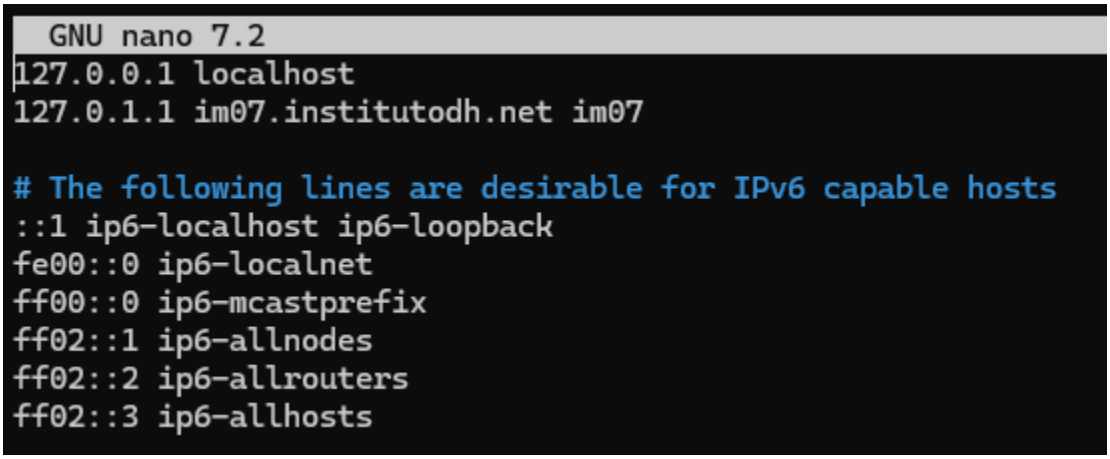
hostnamectl set-hostname im07.institutodh.net

El archivo /etc/hosts

Editamos nano /etc/hosts para asegurar la resolución local de nombres. Esto evita latencias al buscar el propio dominio y garantiza que los contenedores identifiquen la máquina correctamente.

127.0.0.1 localhost

127.0.1.1 im07.institutodh.net im07



Primer Bloque: Jellyfin, Owncast y recorder

En esta primera etapa del despliegue, configuraremos el núcleo de nuestra plataforma multimedia y de emisión. Utilizaremos tres servicios interconectados que permiten no solo consumir contenido almacenado, sino generar y grabar emisiones en tiempo real.

- **Jellyfin**[1]: Actúa como nuestro servidor multimedia. Se encarga de organizar, catalogar y reproducir la biblioteca. Gracias al mapeo de directorios podremos gestionar fácilmente el contenido.
- **Owncast**[2]: servidor de streaming de video auto-alojado (similar a una versión privada de Twitch).
    - **Emisión en Vivo:** Permite recibir señales de video (vía RTMP en el puerto 1935) y distribuirlas a través de un reproductor web integrado. Además, cuenta con chat.
    - **Ajuste de Puertos:** Se ha cambiado el puerto web al 8081 para evitar conflictos de red con otros servicios de videoconferencia (Jitsi).
- **recorder**: es un contenedor (basado en un *build* personalizado)[3] cuyo único propósito es grabar la retransmisión en directo de Owncast y guardarlo en un directorio de Jellyfin para poder visionarlas en otro momento.
    - **Grabación Automática**: Monitoriza la actividad de Owncast y captura la retransmisión en tiempo real.
    - **Sincronización de Datos**: Al compartir el volumen "./biblioteca/Grabaciones_Vivas", el archivo de video generado por este contenedor es depositado directamente en la ruta que Jellyfin escanea, permitiendo que se guarden y estén de manera automática las retransmisiones una vez terminadas.
    - **Dependencia Estructural**: Está configurado para iniciarse solo cuando Owncast esté operativo (**depends_on**).

[1] https://jellyfin.org/docs/general/installation/container/
[2] https://owncast.online/quickstart/container/
[3] https://github.com/teklynk/owncast_stream_recorder

Resumen de redes:

cole_net: Una red privada tipo *bridge* para que el grabador y Owncast hablen entre sí sin exposición innecesaria.

nginx_network: Jellyfin y Owncast la usarán para conectar más adelante con un contenedor Nginx Proxy Manager, el cual se encargará de los certificados y la salida a internet.

MUY IMPORTANTE. En los docker-compose.yml veremos como "**nginx_network**" está definida como "**external: true**". Estos porque por defecto, cuando haces un "docker-compose up", Docker crea una red aislada solo para los servicios de ese archivo. Si queremos que un tercer proyecto (Nginx Proxy Manager) llegue a ambos, necesitamos un "puente" **que ya exista en el sistema**.

Por eso, antes de intentar levantar cualquiera de los tres bloques de contenedores, hay que crear esta red:

docker network create nginx_network

Archivo: \video_streaming\jellyfin_owncast**docker-compose.yml**

Vamos a desgranarlo:

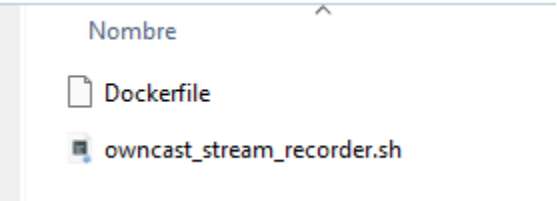
```
services:
 jellyfin:
  image: jellyfin/jellyfin
  container_name: jellyfin
  volumes:
   - ./jellyfin_config:/config
   # Mapeo detallado por tipo de contenido
   - ./biblioteca/Peliculas:/data/peliculas
   - ./biblioteca/Series:/data/series
   - ./biblioteca/Musica:/data/musica
   - ./biblioteca/Libros:/data/libros
   - ./biblioteca/Fotos_y_Videos:/data/videos_caseros
   - ./biblioteca/Videos_Musicales:/data/videos_musicales
   # El "recorder" escribirá aquí, y Jellyfin lo leerá como parte de la biblioteca
   - ./biblioteca/Grabaciones_Vivas:/data/grabaciones_directos
   # Directorio para los archivos .m3u
   - ./biblioteca/live_files:/data/live_files
   # Directorio para las videoconferencias
   - ./biblioteca/videoconferencias:/data/videoconferencias
  ports:
   - "8096:8096"
  networks:
   - nginx_network # Para poder conectar con Nginx Proxy Manager
  restart: unless-stopped
  extra_hosts:
      - "live.im07.institutodh.net:host-gateway"
# "extra_hosts"→ Mapea el dominio a la IP interna del servidor, evitando que Docker se bloquee al intentar salir a internet para conectar consigo mismo (Loopback).
```

```
  owncast:
    image: gabekangas/owncast:latest
    container_name: owncast
    ports:
      - "1935:1935" # Este puerto es por donde recibe las señales de video.
      - "8081:8080" # Cambiamos puerto externo a 8081 para no pisar al contenedor VB
(Jitsi Videobridge)
    volumes:
      - ./owncast_data:/app/data
    networks:
      - nginx_network # Para poder conectar con Nginx Proxy Manager
      - cole_net

  recorder:
    build: ./recorder
    container_name: owncast_recorder
    depends_on:
      - owncast
    volumes:
      # El grabador guarda los archivos directamente donde Jellyfin los busca
      - ./biblioteca/Grabaciones_Vivas:/recordings
    networks:
      - cole_net
    restart: always

networks:
  cole_net:
    driver: bridge
  nginx_network:
    external: true
```

En el directorio de \video_streaming\jellyfin_owncast\ hay una carpeta llamada “recorder” donde se encuentran los archivos necesarios para el contenedor:



- Dockerfile: Servirá para crear el contenedor en sí. Se basa en una imagen de Alpine a la que se le instalará “ffmpeg”, el programa encargado de realizar la grabación de la retransmisión. Las ordenes se las dará el script “owncast_stream_recorder.sh”, del cual tiene orden de copiar y ejecutar.

- “owncast_stream_recorder.sh”[4]:

    - El script ejecuta un loop infinito que:
        - Cada 30 segundos verifica el estado del stream
        - Si el stream está online y no hay grabación activa:
        - Espera 30 segundos (para evitar grabaciones falsas)
        - Verifica nuevamente y si sigue online, inicia la grabación
    - Si el stream está offline y hay grabación activa:
        - Detiene la grabación
        - Gestiona el almacenamiento (mantiene las 6 últimas)
        - Optimiza los videos grabados

[4] https://github.com/teklynk/owncast_stream_recorder

La única parte del script que tenemos que configurar nosotros es la siguiente:

```
# Configuration variables
# --- CONFIGURACIÓN PARA DOCKER ---
# RECUERDA: "owncast" es el nombre del servicio en el docker-compose
API_URL="http://owncast:8080/api/status"
M3U8_STREAM_URL="http://owncast:8080/hls/0/stream.m3u8"
RECORDINGS_DIR="/recordings"
FILE_NAME_PREFIX="colegio_stream_"
```

El borrado a partir de 6 se puede modificar aquí:

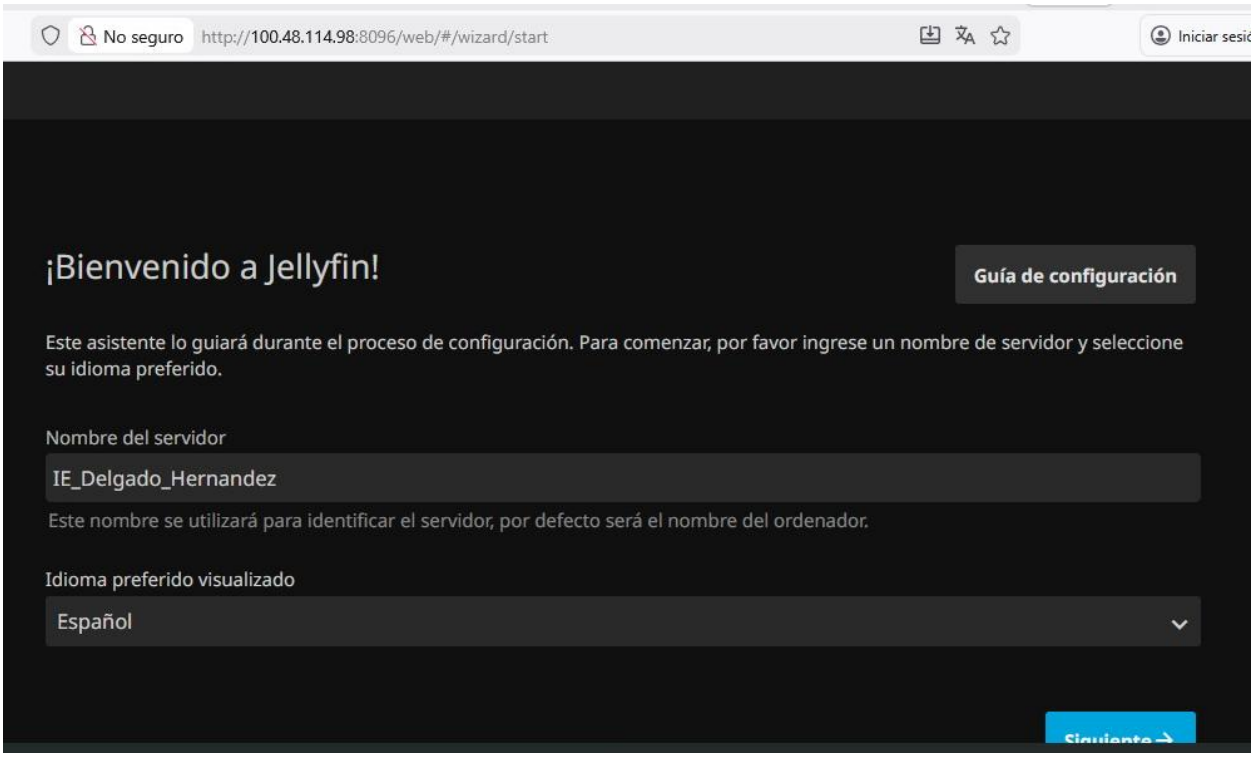
```
if [ ${#recordings[@]} -gt 6 ]; then
    for ((i=6; i<${#recordings[@]}; i++)); do
      echo "Deleting: ${recordings[$i]}"
      rm "${recordings[$i]}"
    done
fi
```

Jellyfin

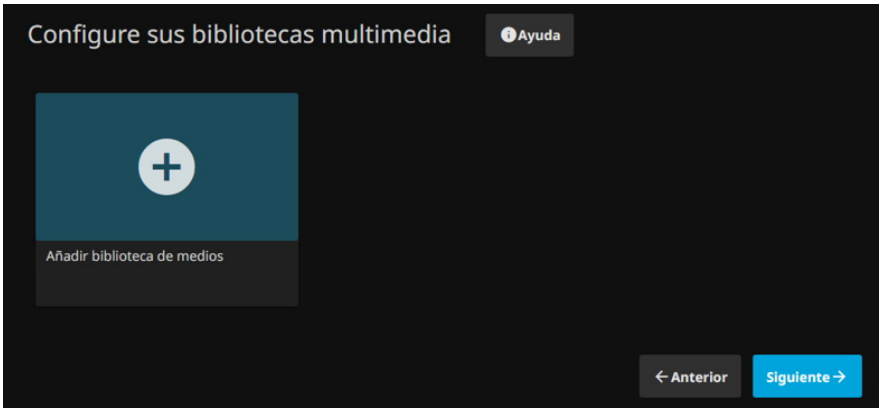
NOTA: Aún no tenemos configurados los subdominios, así que seguiremos usando en la guía las IP y sin certificados.

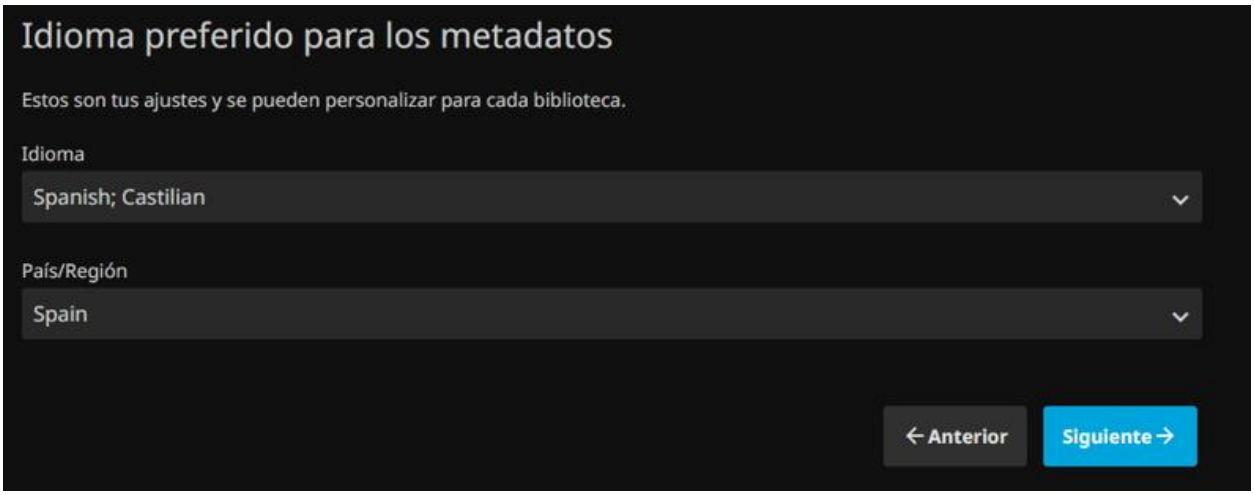
http://100.48.114.98:8096

Al servidor se le genera un nombre automáticamente. Podemos creárselo nosotros.



Podemos configurar ahora o más adelante las bibliotecas: básicamente es elegir una categoría, poner un nombre y especificar el directorio donde estarán los archivos.




Idioma preferido para los metadatos
Estos son tus ajustes y se pueden personalizar para cada biblioteca.
Idioma
Spanish; Castilian
País/Región
Spain
← Anterior
Siguiente →

Importante:


Configurar el acceso remoto
Permitir conexiones remotas a este servidor
Si no está activado, todas las conexiones remotas serán bloqueadas.
← Anterior
Siguiente →


¡Ya está!
Eso es todo lo que necesitamos por ahora. Jellyfin a empezado a recolectar información de su biblioteca. Échale un vistazo a nuestras aplicaciones, y después presione Finalizar para ver el Panel de control.
← Anterior
✓ Terminar

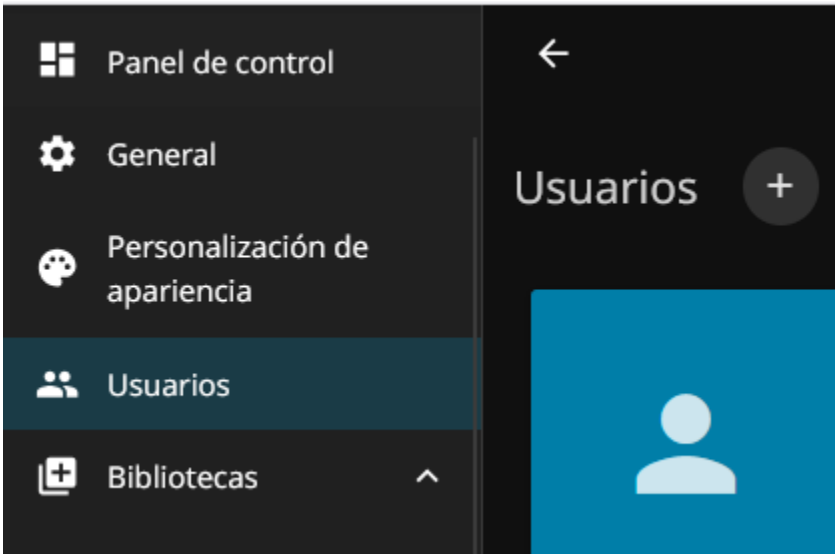
Gestión masiva de usuarios:

Existen varias maneras de hacer esta tarea en Jellyfin, desde usar un Plugin LDAP[5], hasta usar contenedores dedicados a ellos, algunos incluso nos permiten invitar a los usuarios que queramos para que ellos mismos creen sus propias cuentas[6][7].

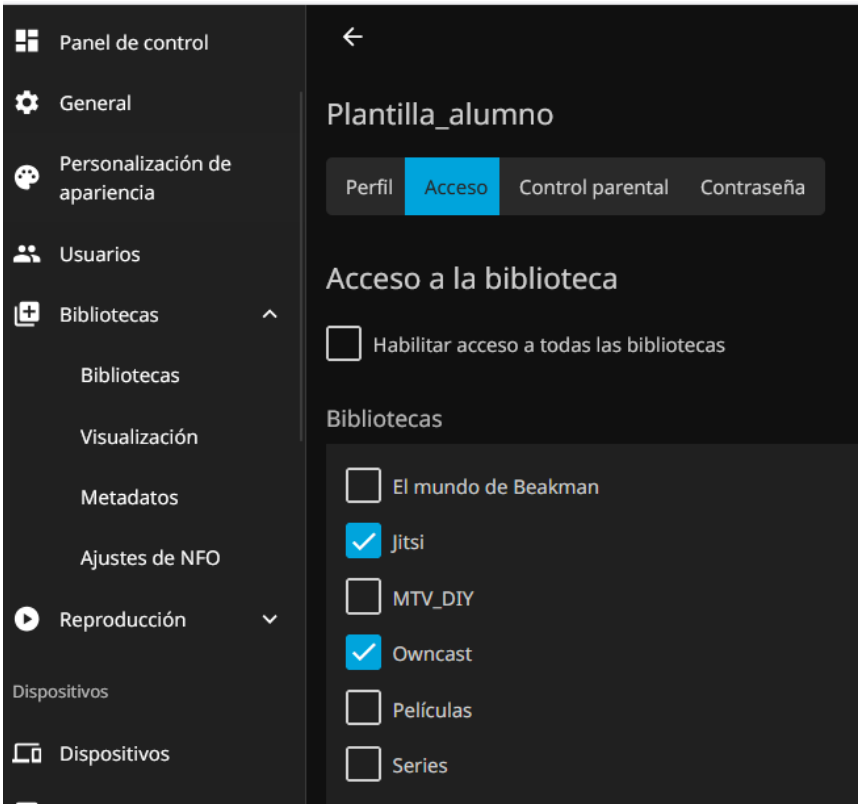
En esta ocasión para simplificar las cosas vamos a usar un script que nos permitirá crear los usuarios de una manera más sencilla, aunque a la larga puede ser más difícil de gestionar.

- En el Panel de Control de Jellyfin:

Lo primero que debemos hacer en Jellyfin es crear un usuario que usaremos como plantilla. Debemos especificar que bibliotecas podrá usar, si tiene acceso a retransmisiones en directo y si podrá descargarse contenido.



[5] https://github.com/jellyfin/jellyfin-plugin-ldapauth
[6] https://github.com/wizarrrr/wizarr
[7] https://jfa-go.com/



Después crearemos una clave API, la cual nos permitirá comunicar el script con nuestro servidor Jellyfin.



- Archivos:

Necesitamos un archivo .csv que contenga los datos de los alumnos. Para el script que tenemos preparado, los datos deben presentarse de esta manera:

Username,Password,Email

m.garcia88,A!9bXz21,m.garcia88@clase.es

javier.ruiz,Kp3#mQ99,javier.ruiz@clase.es

sofia.lopez.a,Lz7$vW42,sofia.lopez.a@clase.es

pablo_mendez,Rj5&nB18,pablo_mendez@clase.es

ana.t.cruz,Qw9!pL33,ana.t.cruz@clase.es

luis_duran,Hx2%kM56,luis_duran@clase.es

Después en el script tendremos que especificar la clave API, la ruta al servidor, el usuario plantilla y el archivo .csv.

API_KEY="376d28bee73649299b91db045a44d32d"

JELLYFIN_URL="http://localhost:8096"

PLANTILLA="Plantilla_alumno"

CSV_FILE="alumnos_2.csv"

El script se encargará primero de recuperar todos los datos del usuario plantilla y los aplicará al nuevo usuario creado.

Se adjuntan junto con este documento los script “crear_usuarios.sh” y “borrar_alumnos.sh”.

Añadir retransmisión en directo:

En el siguiente apartado veremos como configurar la retransmisión en directo con Owncast. Una vez hecha, podemos volver a este punto y realizar la configuración necesaria para poder verla desde Jellyfin.

En nuestro caso hemos creado el archivo “colegio.m3u” en el directorio /biblioteca/live_files/ para el contenedor de Jellyfin.

Su contenido es:

#EXTM3U

#EXTINF:-1 tvg-id="Owncast" tvg-name="Directo Colegio" tvg-logo="https://owncast.online/images/logo.png", Directo Colegio

https://live.im07.institutodh.net/hls/stream.m3u8

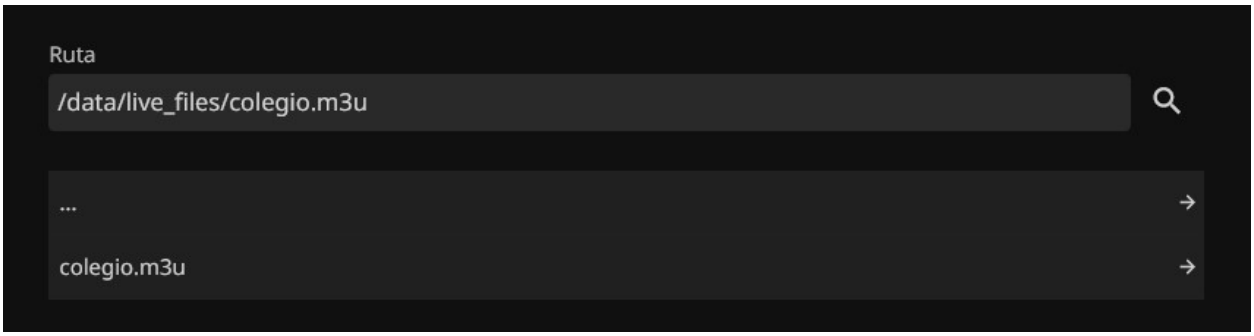
```
 GNU nano 7.2                                  biblioteca/live_files/colegio.m3u
#EXTM3U
#EXTINF:-1 tvg-id="Owncast" tvg-name="Directo Colegio" tvg-logo="https://owncast.online/images/logo.png", Directo Colegio
https://live.im07.institutodh.net/hls/stream.m3u8
```

Ahí debemos colocar la URL de la retransmisión.

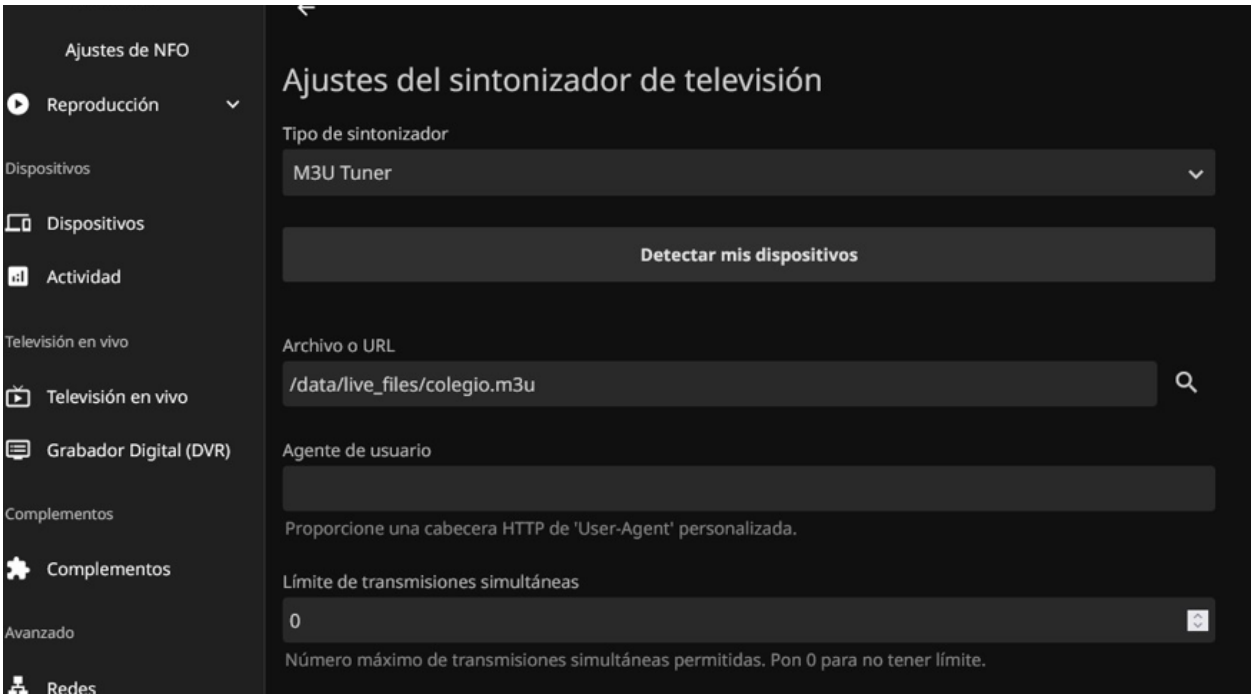
Una vez que tenemos el archivo en el directorio indicado, vamos a la interfaz web de Jellyfin y nos vamos al Panel de Control → Televisión en vivo → Add Tuner Device



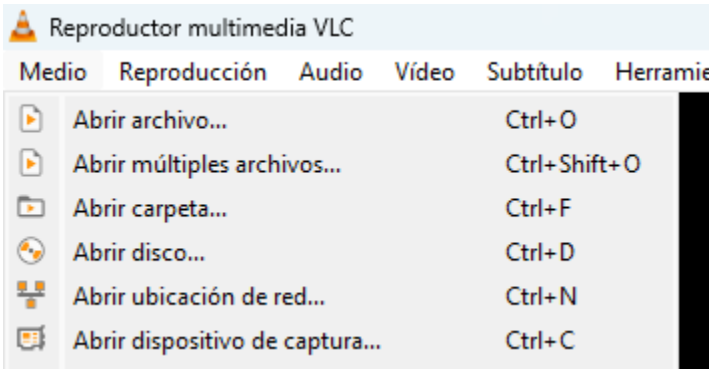
Especificamos el archivo:

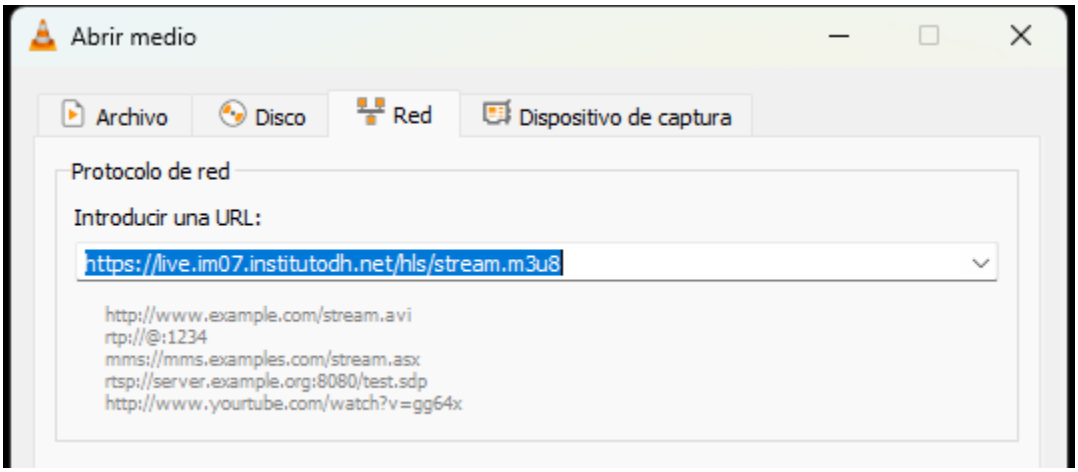


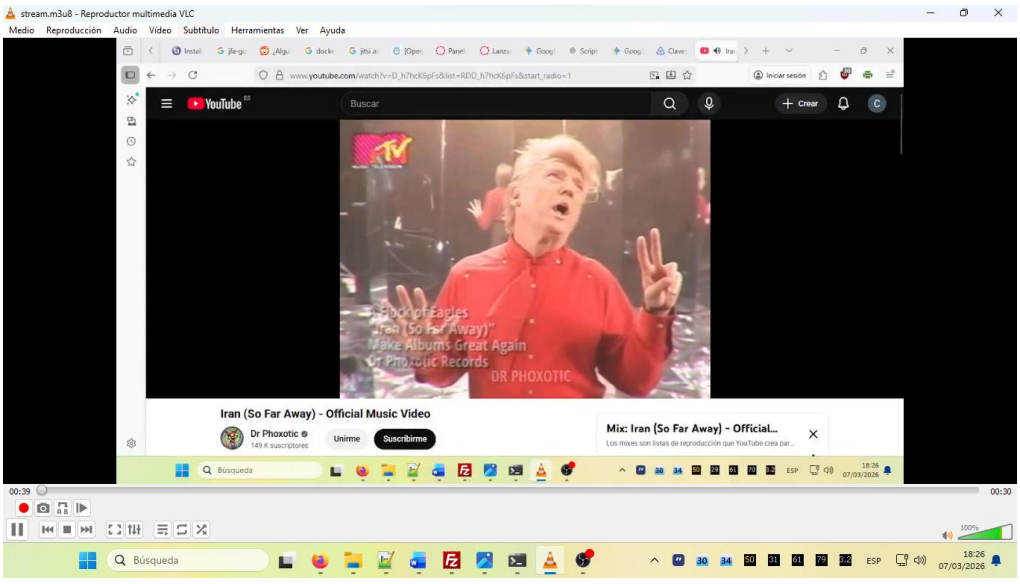
Podemos realizar más configuraciones, pero para nuestro caso las que vienen por defecto funcionan bien.



Con esta URL (https://live.im07.institutodh.net/hls/stream.m3u8) también podemos verlo en VLC: Medio → Abrir ubicación de red.







Ver en otros dispositivos:

Android:

Existe aplicación oficial de Jellyfin en Android.

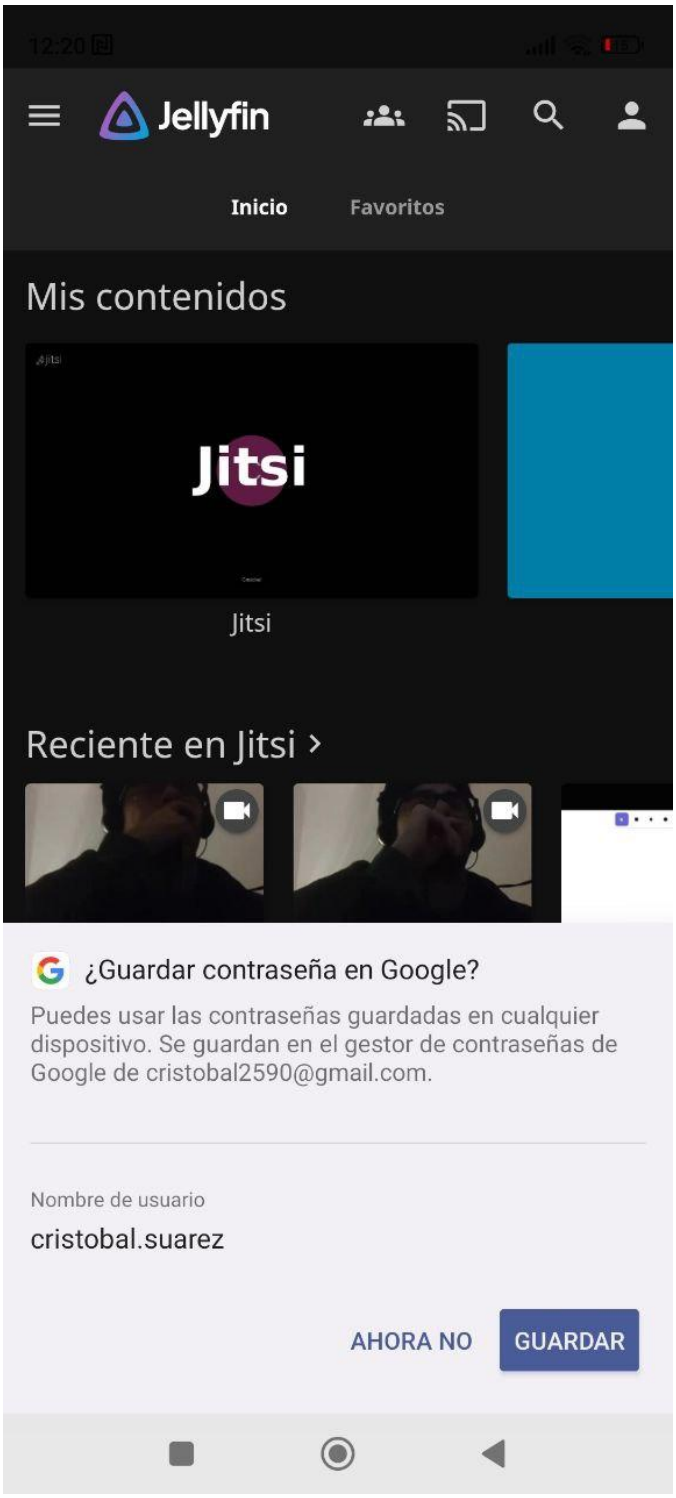
Indicamos el servidor



Credenciales:

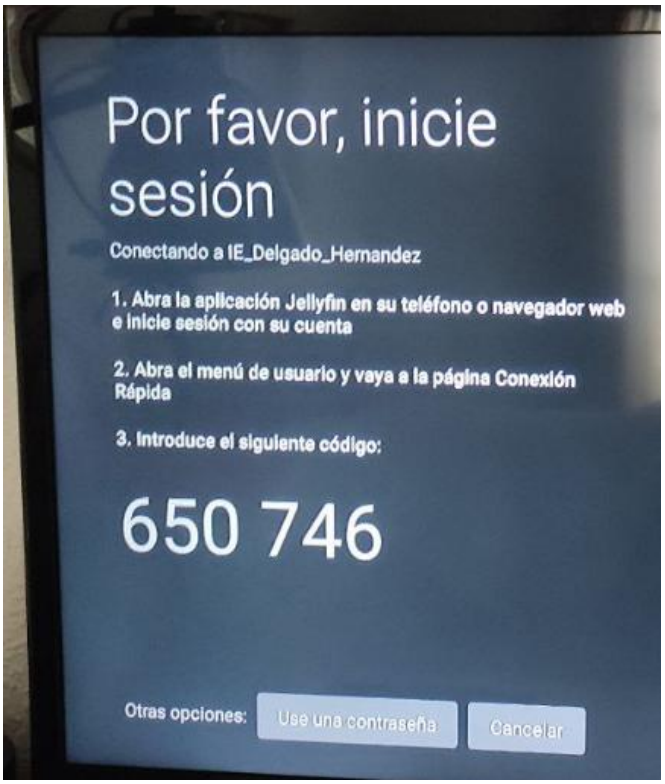


Ya estamos dentro:

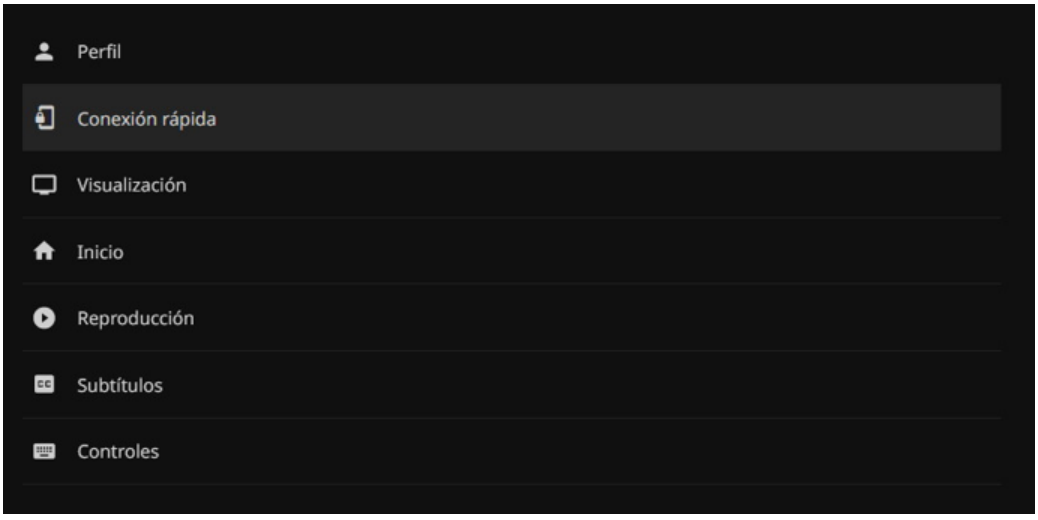


Smart TV

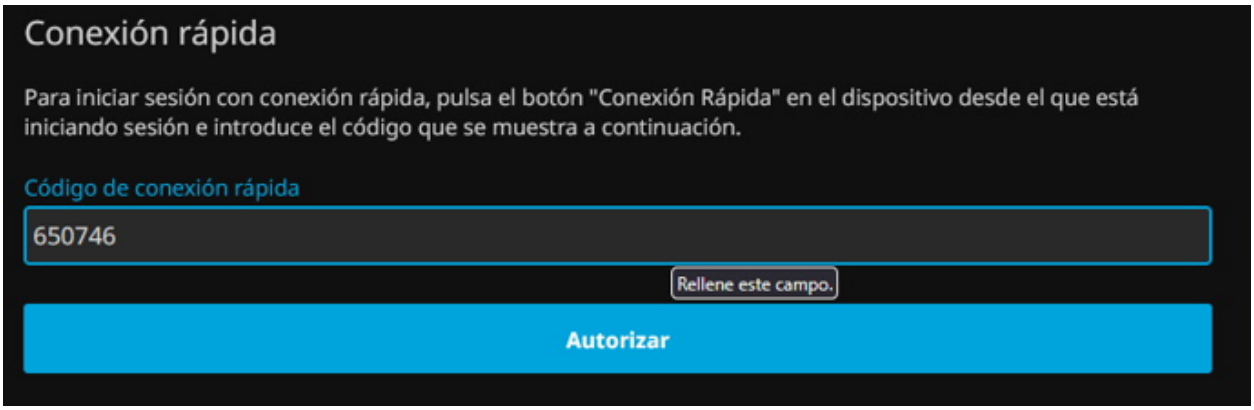
En nuestro caso vamos a conectarnos con una Fire TV. Nos vamos a la App Store y descargamos la aplicación oficial de Jellyfin. De igual manera que en el anterior, ponemos la dirección del servidor y luego nos pedirá credenciales de usuario. En este caso, para que veamos otra manera de introducir credenciales, vamos a usar la de Conexión Rápida:



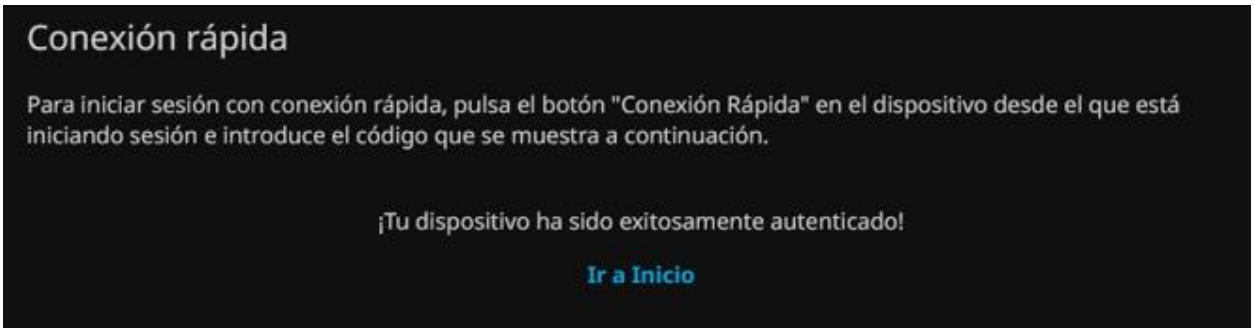
La propia Smart TV nos indica el código antes de que nosotros hayamos hecho nada. Ahora, nos vamos a alguna sesión abierta que tengamos en Jellyfin (PC, Android, etc). Nos vamos a Perfil → Conexión Rápida.

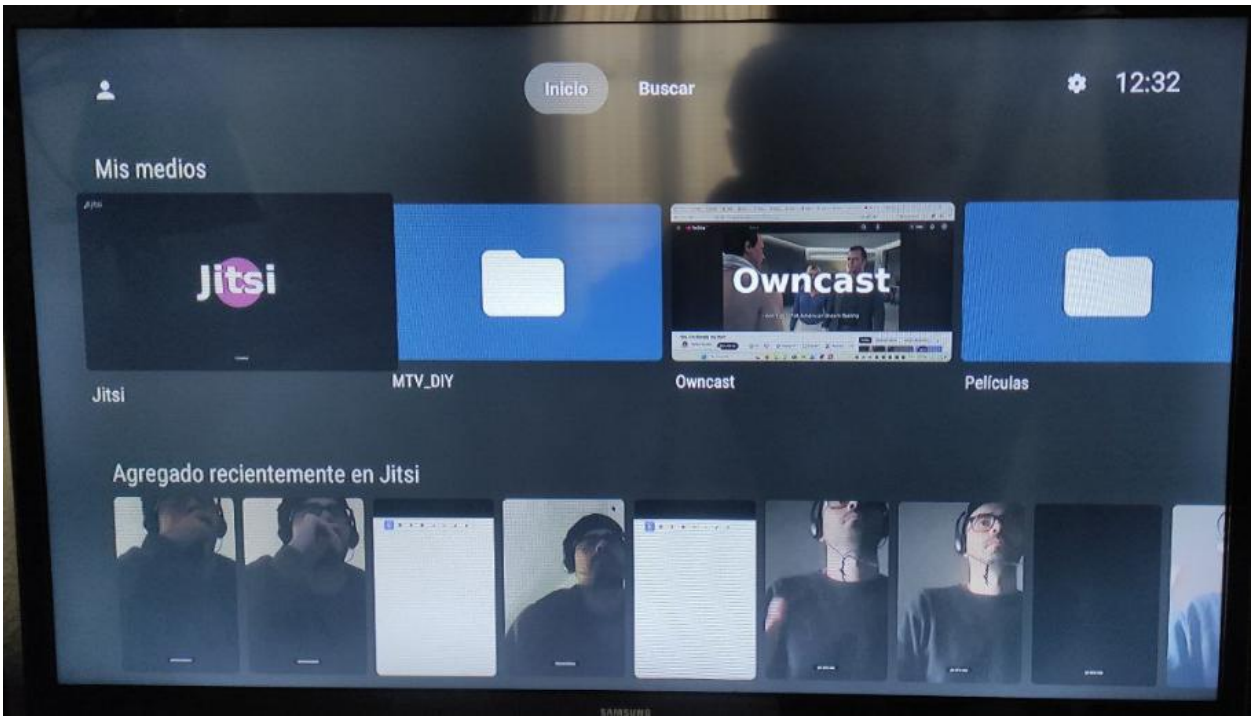


Introducimos el código:



Conectado



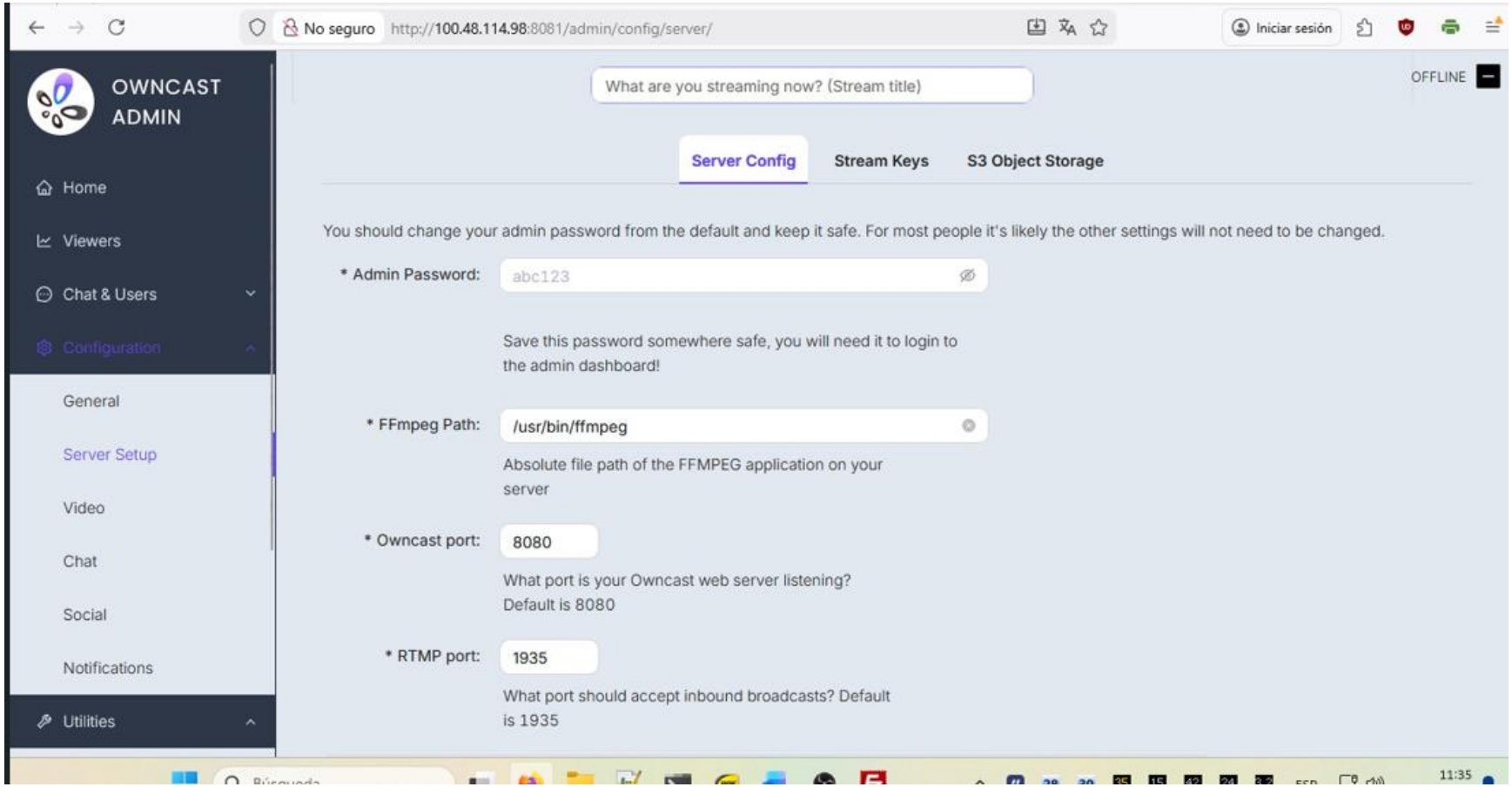


Owncast

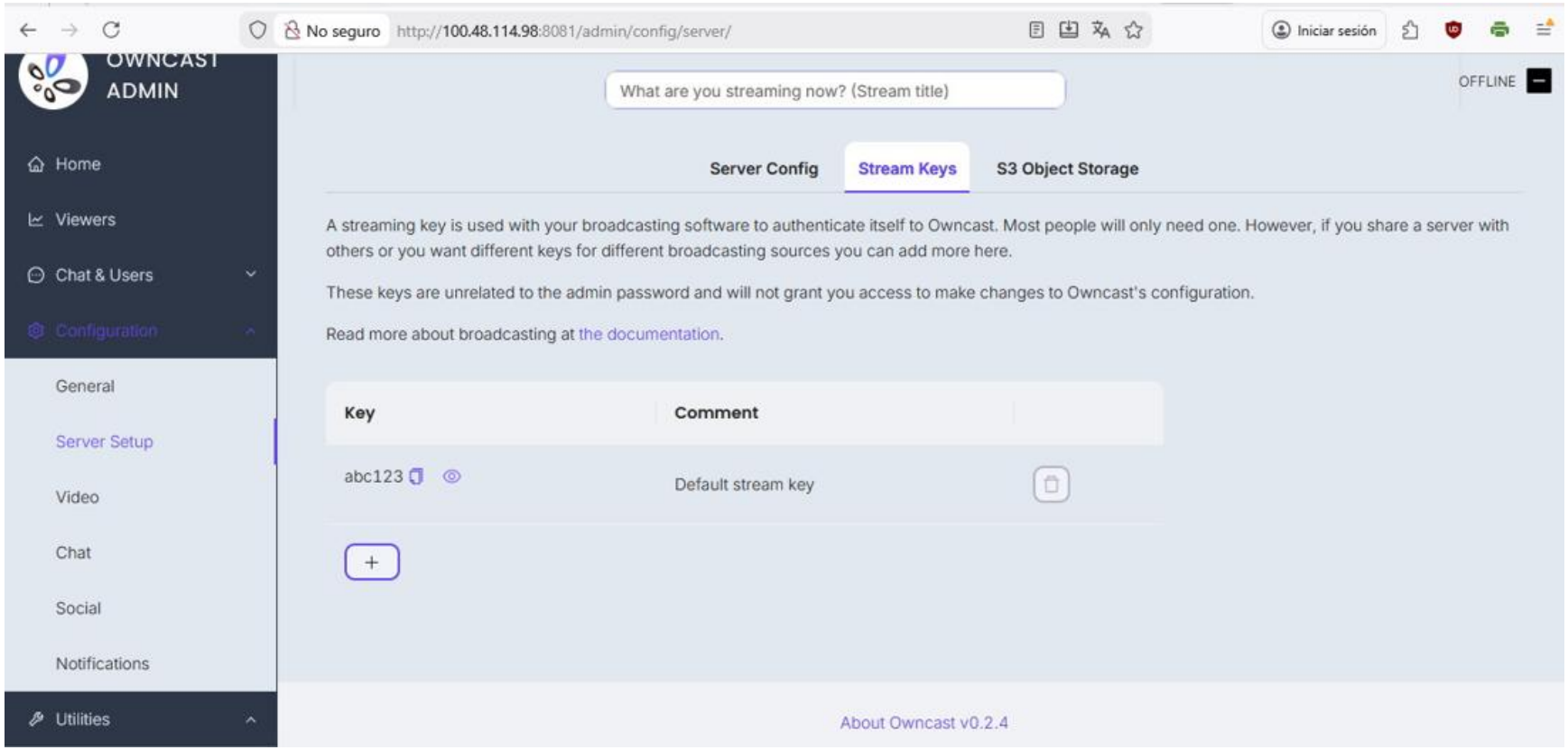
http://100.48.114.98:8081/admin

Entramos primero en la sección de admin. Nos pedirá credenciales para entrar. Por defecto el usuario es “admin” y la contraseña “abc123”.

El primer paso será ir hasta “Server Setup” → “Server Config” y cambiar la contraseña del admin.



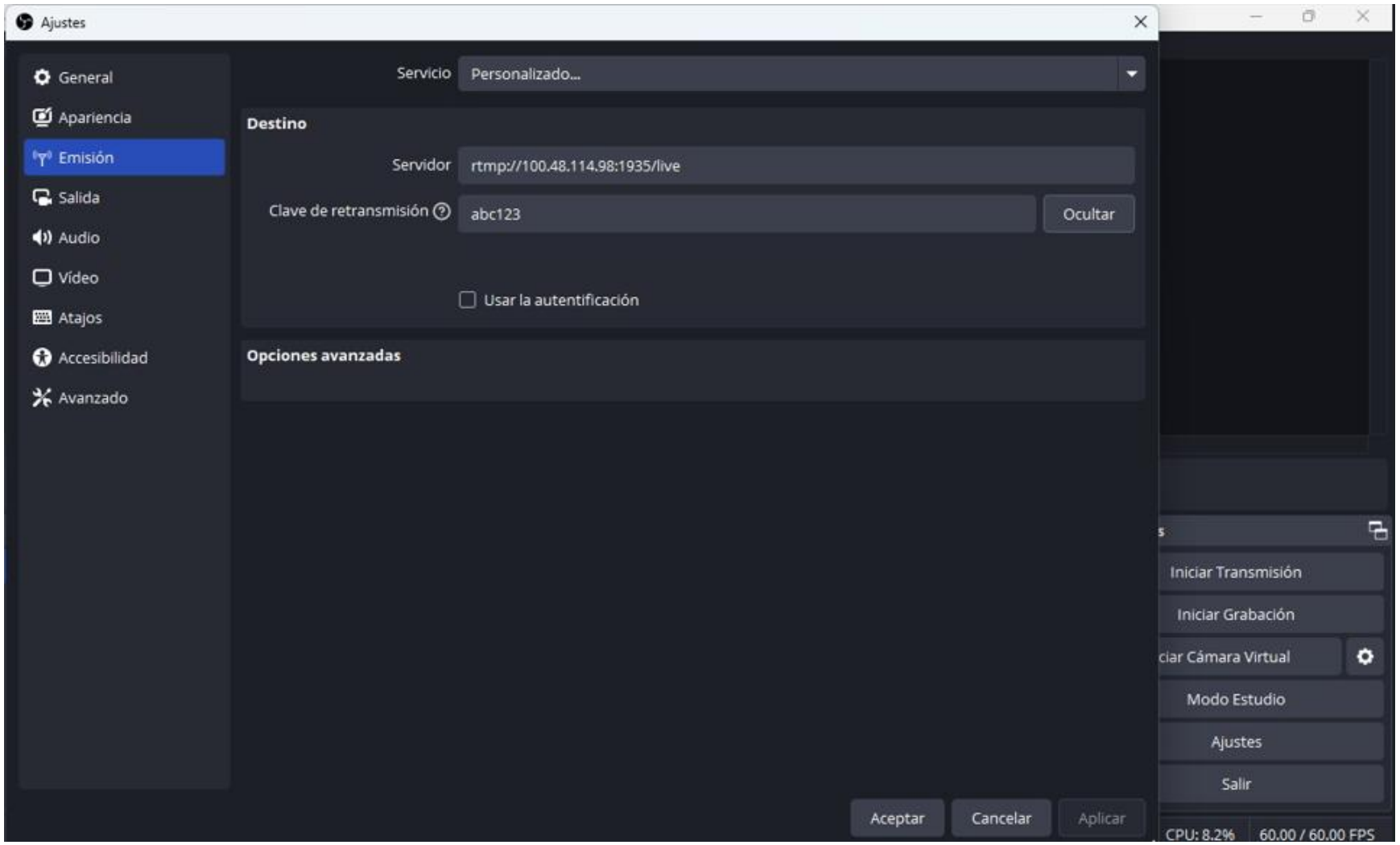
El contenedor está prácticamente listo para retransmitir. Ahora solo debemos ir a la pestaña de “Stream Keys”. Copiamos la “key” o generamos una nueva (seguridad). Por defecto es “abc123”.



Ahora vamos a usar dos métodos de emisión de señal de video que funcionan a la perfección con Owncast.

El primero es OBS Studio. Ajustes → Emisión:

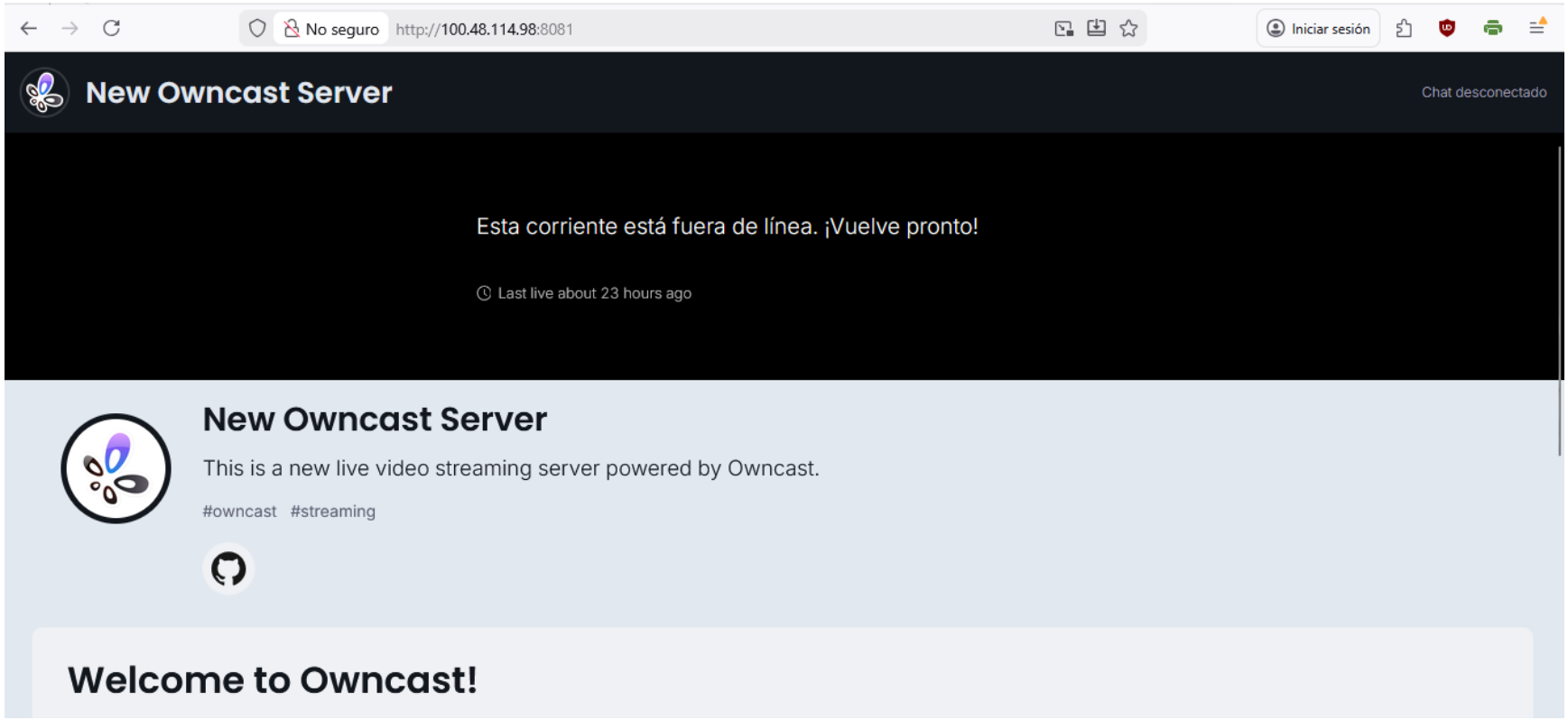
Servidor: **rtmp://100.48.114.98:1935/live** y la clave "**abc123**".



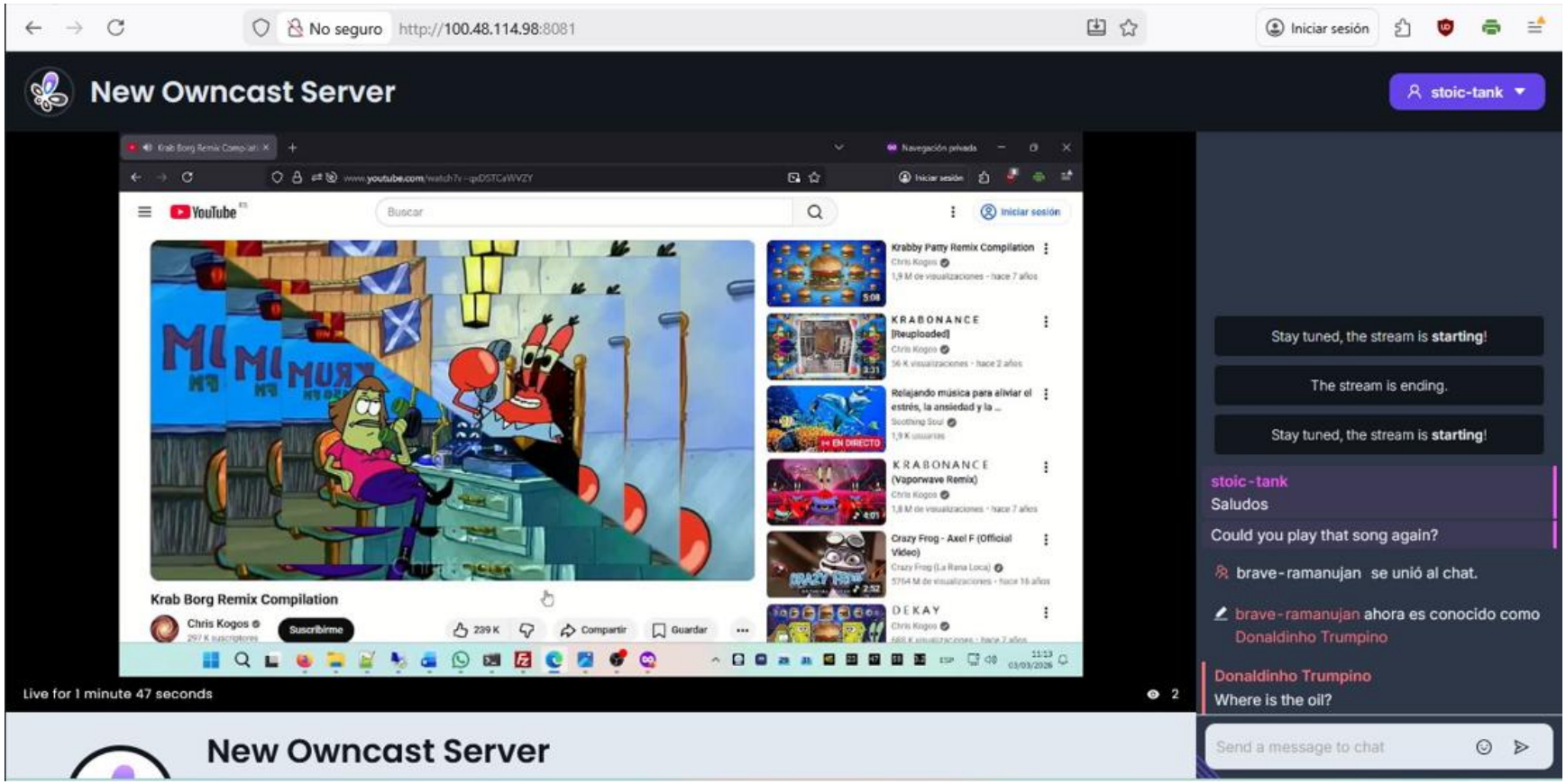
Una vez configurada solo tendremos que darle a “**Iniciar Transmisión**” y se generará la retransmisión. Para verla nos vamos a:

http://100.48.114.98:8081

Cuando no recibe señal veremos esto.



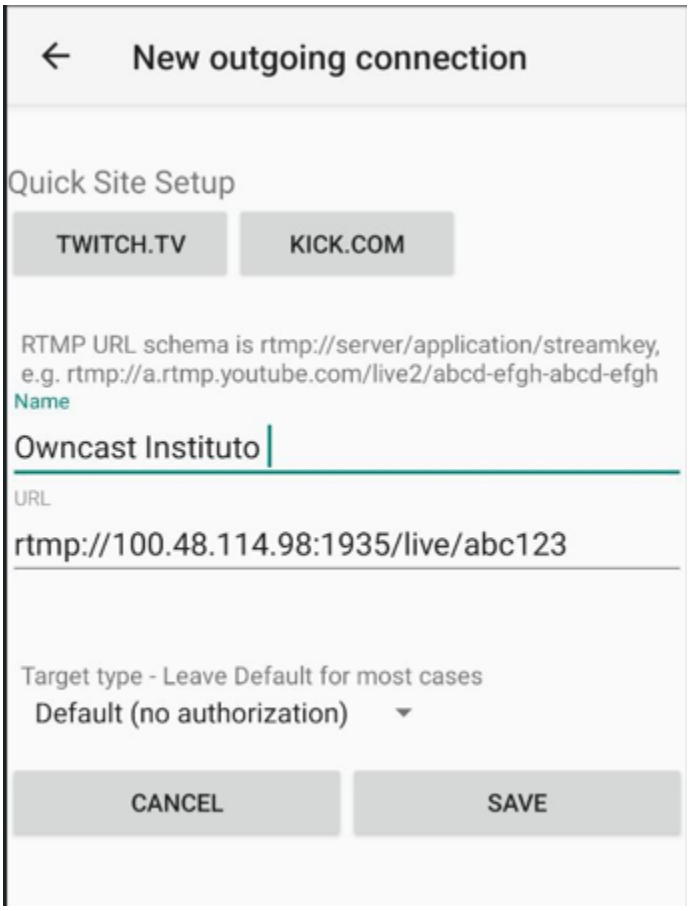
Con la retransmisión ya activa. Y chat.



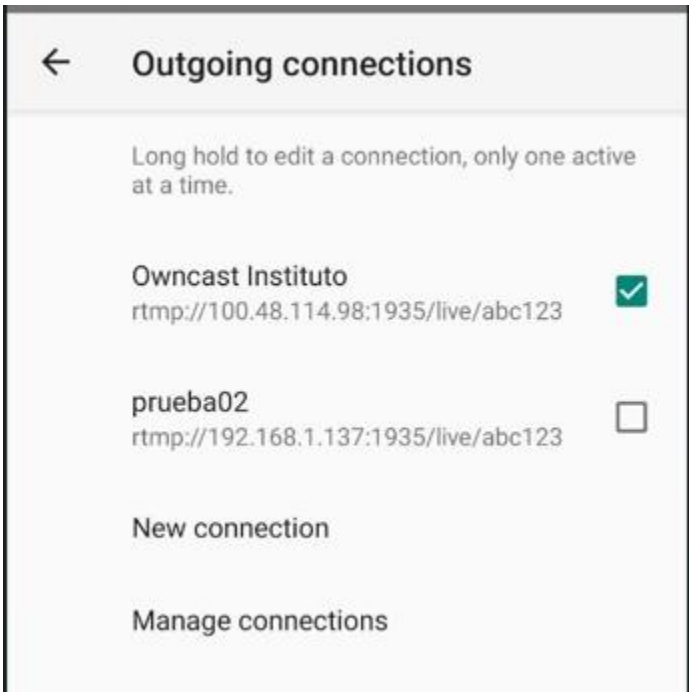
También podemos retransmitir desde dispositivos móviles. En esta ocasión se ha probado desde un móvil Android. La app utilizada es “IRL Pro”.



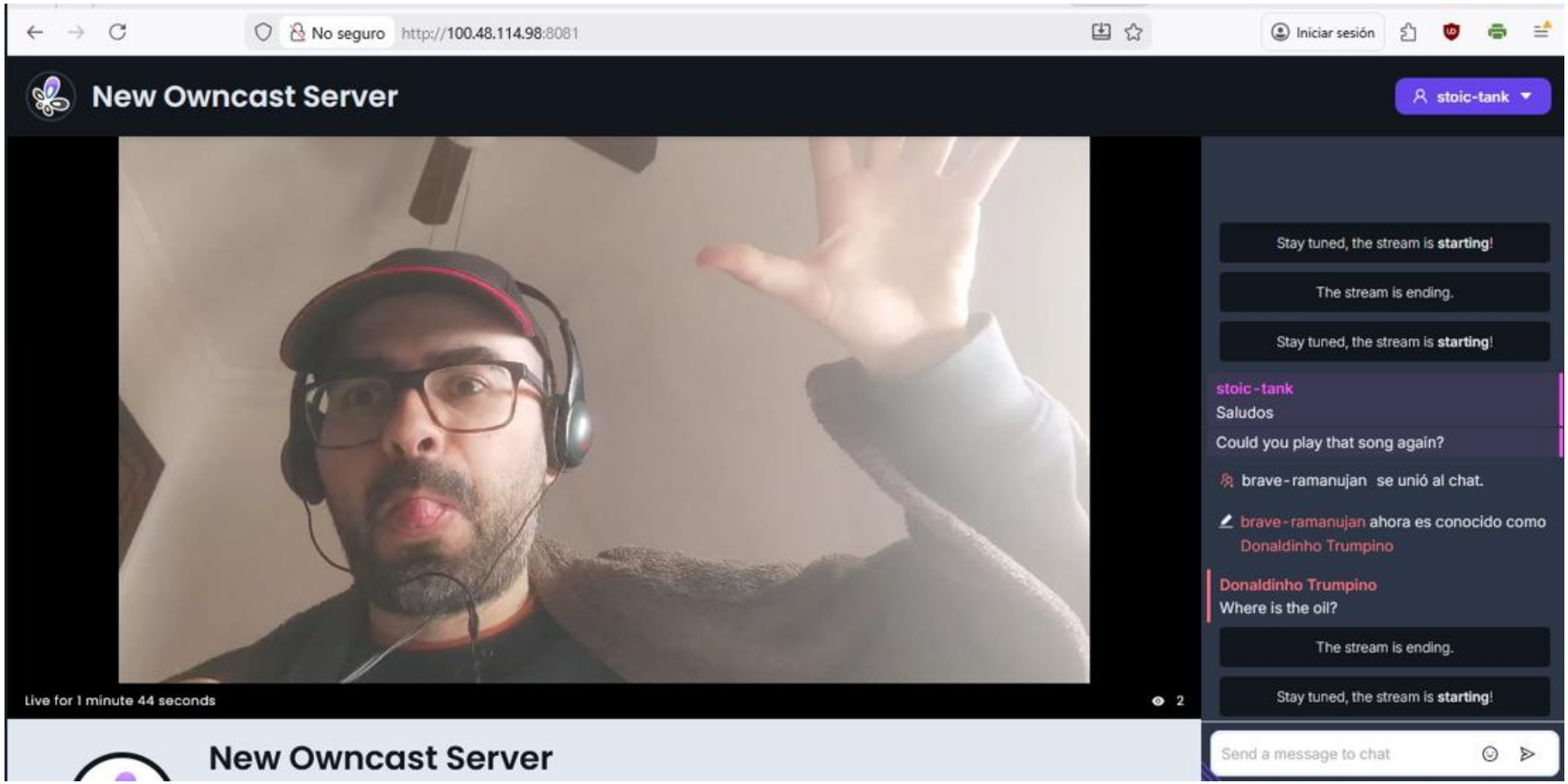
En la app, nos vamos a Ajustes → Connection → New Connection e introducimos los datos necesarios.



En este caso ponemos el token en la propia URL.


Outgoing connections
Long hold to edit a connection, only one active at a time.
Owncast Instituto
rtmp://100.48.114.98:1935/live/abc123
prueba02
rtmp://192.168.1.137:1935/live/abc123
New connection
Manage connections

Después solo tendremos que darle al **play** (nos saldrá una ventana de confirmación) y podremos retransmitir el video.



recorder

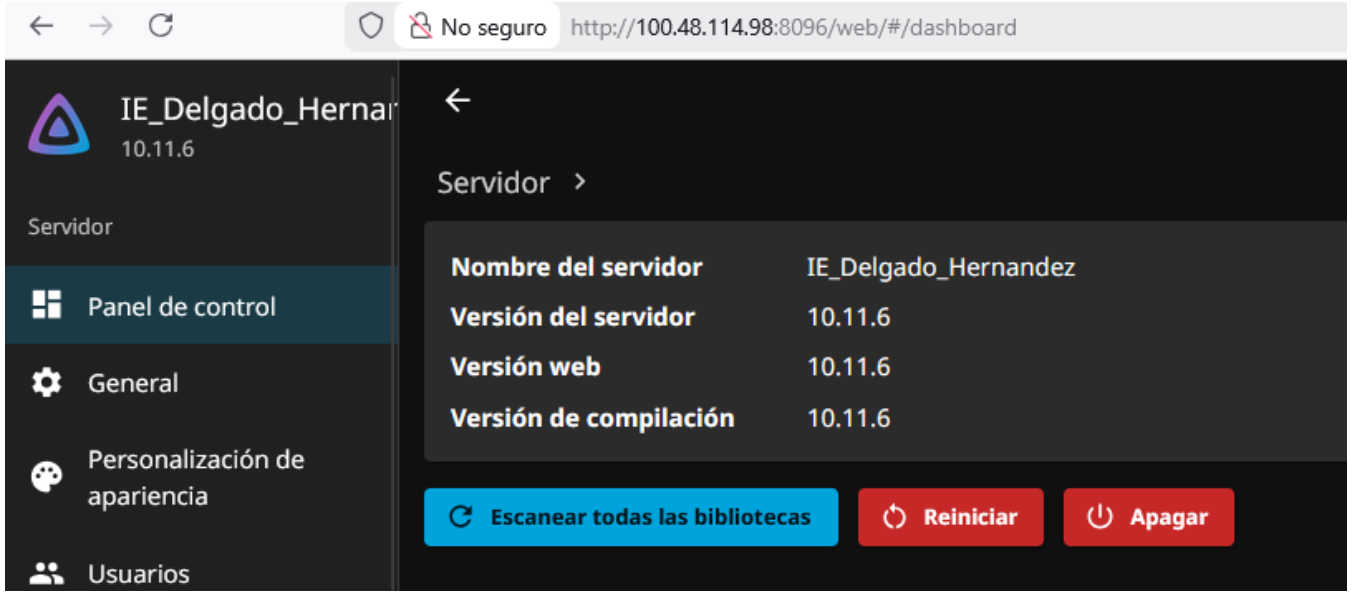
El contenedor “**recorder**” cumple su función de grabar las retransmisiones. Pero recuerda que deben ser como mínimo de 30 segundos. Y además que la señal tiene unos 10 – 15 segundos de retraso.

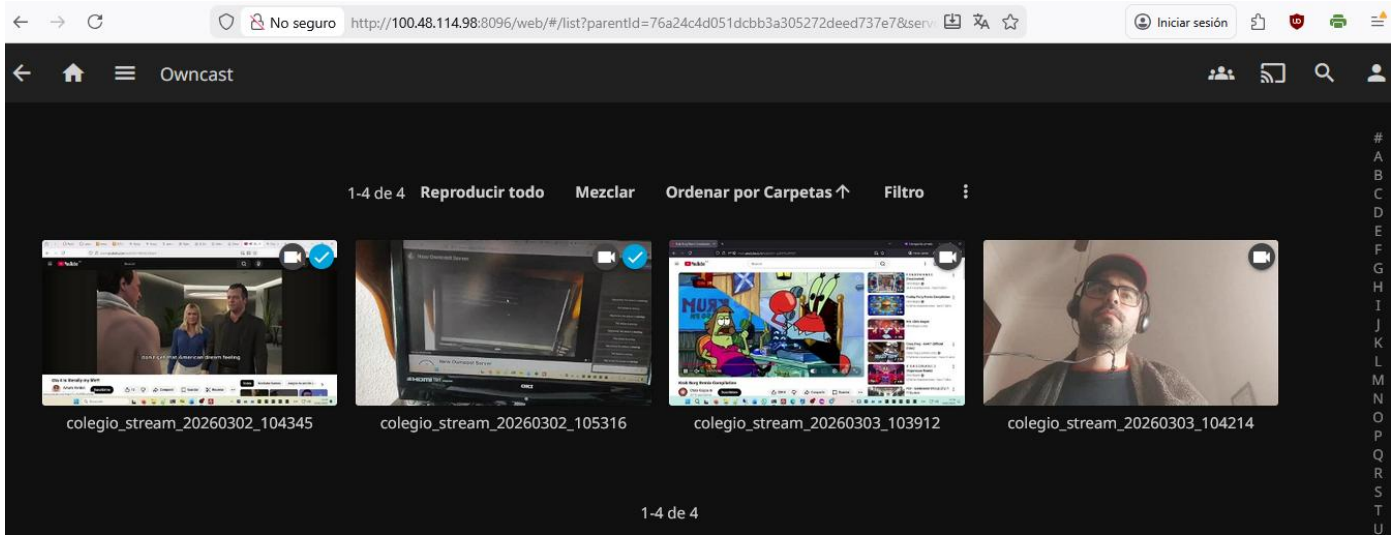
```
root@ip-172-31-42-182:/video_streaming/jellyfin_owncast# docker logs -f owncast_recorder
Script iniciado
Stream is online. Waiting 30 seconds before starting recording...
Starting recording...
ffmpeg version 8.0.1 Copyright (c) 2000-2025 the FFmpeg developers
  built with gcc 15.2.0 (Alpine 15.2.0)
```

Se guardan en el directorio “**Grabaciones_Vivas**”.

```
^Croot@ip-172-31-42-182:/video_streaming/jellyfin_owncast# ls -l biblioteca/Grabaciones_Vivas/
total 19564
-rw-r--r-- 1 root root  4415586 Mar  2 10:53 colegio_stream_20260302_104345.mp4
-rw-r--r-- 1 root root  5033947 Mar  2 10:53 colegio_stream_20260302_105316.mp4
-rw-r--r-- 1 root root 10578901 Mar  3 10:40 colegio_stream_20260303_103912.mp4
root@ip-172-31-42-182:/video_streaming/jellyfin_owncast#
```

De igual manera que veremos con **Jitsi**. Ahora solo tendremos que ir a Jellyfin → Panel de control → Escanear bibliotecas.





Segundo Bloque: Jitsi Meet

Tras configurar los servicios de biblioteca multimedia y retransmisión en directo, este bloque se centra en el de videoconferencia. Jitsi Meet ha sido el software elegido por su versatilidad y facilidad de uso.

A continuación, detallamos el proceso de despliegue mediante Docker, prestando especial atención a la configuración de variables de entorno en el archivo ".env".

Proceso de instalación[8]:

- Descargamos la última versión estable:

wget $(wget -q -O - https://api.github.com/repos/jitsi/docker-jitsi-meet/releases/latest | grep zip | cut -d\" -f4)

- Lo descomprimimos:

unzip "nombre_archivo"

- Nos introducimos en el directorio que acabamos de descomprimir y creamos un archivo ".env" a partir de "env.example".

cp env.example .env

- Generamos contraseñas fuertes en el archivo ".env" ejecutando el siguiente script:

./gen-passwords.sh

- Creamos los directorios para CONFIG:

mkdir -p ~/.jitsi-meet-cfg/{web,transcripts,prosody/config,prosody/prosody-plugins-custom,jicofo,jvb,jigasi,jibri}

NOTA: La creación de estos directorios puede que no sea necesaria según donde se vayan a localizar. En nuestro caso, como veremos más adelante, vamos a crearlos en la propia carpeta donde se ha descomprimido el archivo anterior.

[8] https://jitsi.github.io/handbook/docs/devops-guide/devops-guide-docker/

OTRA NOTA: En algunas guías[9] se menciona la necesidad de habilitar el módulo "**sudo modprobe snd-aloop**" para solucionar ciertos problemas con las grabaciones (contenedor Jibri). Durante la realización de esta guía si me hizo falta instalarlo cuando estaba probando en una VM Ubuntu dentro de Proxmox; pero no me hizo falta hacerlo en otra VM Ubuntu en AWS. Es más, en AWS no pude instalarlo. De todas maneras, aquí pongo la manera de activarlo.

sudo modprobe snd-aloop

- Para que no se borre al reiniciar el PC:

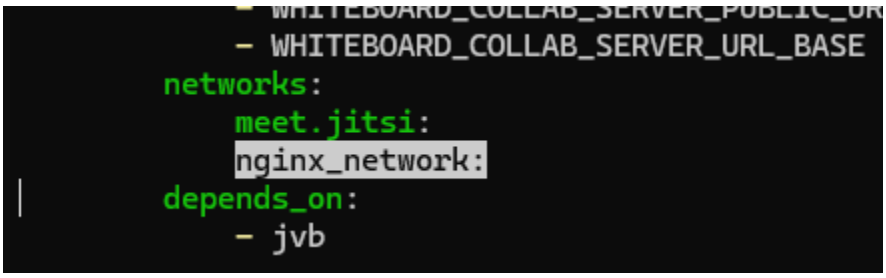
echo "snd-aloop" | sudo tee -a /etc/modules

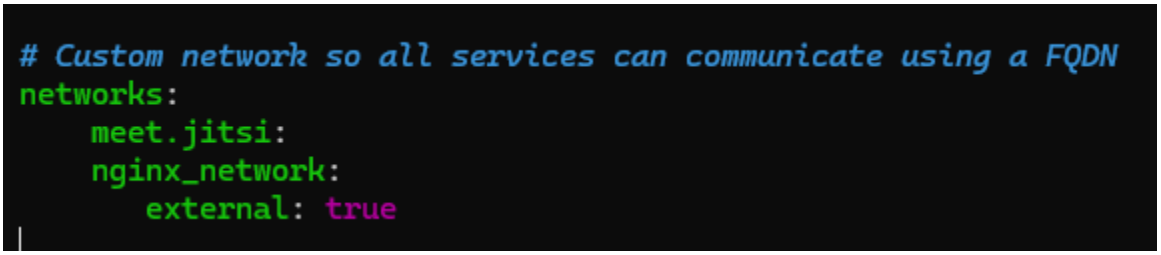
[9] https://oneuptime.com/blog/post/2026-01-15-install-jitsi-meet-ubuntu/view

Configurar archivos

En esta ocasión apenas vamos a tener que configurar los archivos docker-compose.yml que nos vienen por defecto. Solo el principal y el de "**jibri.yml**". El resto de la configuración la haremos en el archivo "**.env**".

En "**docker-compose.yml**" debemos incluir la red para que podamos usar el **Nginx Proxy Manager**. Tanto en la configuración del contenedor Web, como al final del documento.





Después en "**jibri.yml**" vamos a mapear el directorio donde el contenedor guarda las videoconferencias y vamos a hacer que vayan a un directorio que creamos previamente en el contenedor de Jellyfin. De esta manera estarán disponible para su visionado.



Configuración de .env

- Directorio donde se guardan los archivos de CONFIG: En este caso, he decidido que se guarden en el mismo directorio donde se encuentra el “docker-compose.yml”.

```
# Directory where all configuration will be stored
CONFIG=./jitsi-config
```

- Puertos: Los puertos que están comentados se usan ANTES de la configuración con Nginx Proxy Manager. Los otros son para después. En realidad, el HTTPS no es necesario una vez que se configura NPM.

```
# Exposed HTTP port (will redirect to HTTPS port)
#HTTP_PORT=8000
HTTP_PORT=80

# Exposed HTTPS port
#HTTPS_PORT=8443
HTTPS_PORT=443
```

- PUBLIC_URL: Esto le dice a Jitsi exactamente qué dirección deben usar los navegadores para conectar. Sin esto, el sistema intentaría usar nombres internos que no funcionan en internet.

```
# Public URL for the web service (required)
# Keep in mind that if you use a non-standard HTTPS port, it has to appear in the public URL
#PUBLIC_URL=https://meet.example.com:${HTTPS_PORT}
# VITAL PARA EVITAR FALLOS DE CONEXIÓN EN VIDEOCONFERENCIA
#PUBLIC_URL=https://100.48.114.98:8443
PUBLIC_URL=https://meet.im07.institutodh.net
```

- JVB_ADVERTISE_IPS y DOCKER_HOST_ADDRESS: Si no "anuncian" la IP real, los participantes se verán a sí mismos, pero no a los demás (pantallas negras), ya que el tráfico de video no sabría a dónde ir.

```
#JVB_ADVERTISE_IPS=192.168.1.1,1.2.3.4,192.168.178.1#12000,fe80::1#12000
# VITAL PARA EVITAR FALLOS DE CONEXIÓN EN VIDEOCONFERENCIA
JVB_ADVERTISE_IPS=100.48.114.98
# Añade esta línea si no la tienes para ayudar en redes complejas
DOCKER_HOST_ADDRESS=100.48.114.98
```

- Let's Encrypt: Deshabilitamos “Let's Encrypt” porque de ello se encargará Nginx Proxy Manager, no Jitsi.

```
# Enable Let's Encrypt certificate generation
#ENABLE_LETSENCRYPT=1
ENABLE_LETSENCRYPT=0
```

- ETHERPAD: Documento compartido. Nos permitirá mostrar un documento durante la videoconferencia donde los participantes podrán escribir.
    - ETHERPAD_URL_BASE: Configurado para que los contenedores se hablen internamente.
    - ETHERPAD_PUBLIC_URL: permite que los usuarios escriban notas en tiempo real durante la reunión.

```
# Etherpad integration
ETHERPAD_URL_BASE=http://etherpad.meet.jitsi:9001
#ETHERPAD_PUBLIC_URL=https://100.48.114.98:8443/etherpad/p/
ETHERPAD_PUBLIC_URL=https://meet.im07.institutodh.net/etherpad/p/
```

- WHITEBOARD: Nos permite tener una pizarra virtual colaborativa.
    - WHITEBOARD_ENABLED=1: Habilita la pizarra.
    - WHITEBOARD_COLLAB_SERVER_URL_BASE: Configurado para que los contenedores se hablen internamente.
    - WHITEBOARD_COLLAB_SERVER_PUBLIC_URL: Mapeado a través del puerto 8443, permitiendo que los usuarios escriban notas en tiempo real durante la reunión.

```
# Whiteboard integration
WHITEBOARD_ENABLED=1

# Cambia el 3000 por el 80 (puerto interno del contenedor)
WHITEBOARD_COLLAB_SERVER_URL_BASE=http://whiteboard.meet.jitsi:80
#WHITEBOARD_COLLAB_SERVER_PUBLIC_URL=https://100.48.114.98:8443/excalidraw-backend
WHITEBOARD_COLLAB_SERVER_PUBLIC_URL=https://meet.im07.institutodh.net/excalidraw-backend
```

- Autenticación: Jitsi ofrece múltiples maneras de autenticarse[10]: interna, LDAP, JWT tokens, Matrix, Hybrid Matrix Token y externa. Para nuestro caso, hemos elegido interna.
    - ENABLE_AUTH=1: Ahora el sistema requiere un usuario.
    - ENABLE_GUESTS=1: Permite que invitados entren a la reunión, pero solo si un anfitrión autenticado los deja pasar (sistema de lobby).
    - AUTH_TYPE=internal: Se usará el tipo de autenticación interna.
    - ENABLE_AUTO_LOGIN=0 Habilita/Deshabilita el autologin. Para esta práctica lo he dejado en 0 para poder mostrarlo en la presentación.
    - JICOFO_AUTH_LIFETIME=24 hours: Especifica el tiempo para la finalización de sesión de un usuario autenticado.

```
#
# Authentication configuration (see handbook for details)
#

# Enable authentication (will ask for login and password to join the meeting)
ENABLE_AUTH=1

# Enable guest access (if authentication is enabled, this allows for users to be held in lobby until registered user lets them in)
ENABLE_GUESTS=1

# Select authentication type: internal, jwt, ldap or matrix
AUTH_TYPE=internal

#Enable auto login. 0 Disabled. 1 Enabled
ENABLE_AUTO_LOGIN=0

#Select session timeout value for an authenticated user
JICOFO_AUTH_LIFETIME=24 hours
```

[10] https://jitsi.github.io/handbook/docs/devops-guide/devops-guide-docker/

- Jibri/Grabaciones: Las grabaciones se manejan con el contenedor Jibri. Para habilitarlo:
    - ENABLE_RECORDING=1
    - XMPP_SERVER=xmpp.meet.jitsi: Jibri necesita saber dónde está el servidor XMPP internamente
    - JIBRI_RECORDING_DIR=/recordings: Directorio de grabaciones (interno del contenedor).
    - En esta ocasión he usado una serie de opciones durante la prueba en un equipo con IP Privada; las cuales fueron necesarias para poder llevar a cabo las grabaciones. Permiten ignorar errores de certificado para que Jibri pueda entrar por IP:
        - XMPP_TRUST_ALL_CERTS=true
        - JIBRI_HTTP_API_EXTERNAL_TLS=false
        - IGNORE_CERTIFICATE_ERRORS=true

```
# Enable recording
ENABLE_RECORDING=1
# Jibri necesita saber dónde está el servidor XMPP internamente
XMPP_SERVER=xmpp.meet.jitsi
# Directorio de grabaciones (asegúrate de que exista en tu home) / Esto es un directorio interno
#JIBRI_RECORDING_DIR=/config/recordings #ANTES DEL MAPEO CON JELLYFIN
JIBRI_RECORDING_DIR=/recordings

# ESTO LO HACEMOS CUANDO USAMOS JIBRI EN LOCAL Y SIN CERTIFICADO
# Ignorar errores de certificado para que Jibri pueda entrar por IP
XMPP_TRUST_ALL_CERTS=true
JIBRI_HTTP_API_EXTERNAL_TLS=false
IGNORE_CERTIFICATE_ERRORS=true
```

Gestión de usuarios:

Para la gestión de usuarios se usa el contenedor de Prosody de Jitsi.

- Si queremos usar los comandos desde fuera del contenedor:
    - Desde el directorio de Jitsi:

Ver usuarios:

docker exec jitsi-docker-jitsi-meet-35879bb-prosody-1 find /config/data/meet%2ejitsi/accounts -type f -exec basename {} .dat \;

Añadir usuario:

docker exec jitsi-docker-jitsi-meet-35879bb-prosody-1 prosodyctl --config /config/prosody.cfg.lua register TheDesiredUsername meet.jitsi TheDesiredPassword

Borrar usuario:

docker exec jitsi-docker-jitsi-meet-35879bb-prosody-1 prosodyctl --config /config/prosody.cfg.lua unregister TheDesiredUsername meet.jitsi

- También podemos hacerlo desde dentro del contenedor[11]:

Internal users must be created with the prosodyctl utility in the prosody container. In order to do that, first, execute a shell in the corresponding container:

docker compose exec prosody /bin/bash

Once in the container, run the following command to create a user:

prosodyctl --config /config/prosody.cfg.lua register TheDesiredUsername meet.jitsi TheDesiredPassword

Note that the command produces no output.

To delete a user, run the following command in the container:

prosodyctl --config /config/prosody.cfg.lua unregister TheDesiredUsername meet.jitsi

To list all users, run the following command in the container:

find /config/data/meet%2ejitsi/accounts -type f -exec basename {} .dat \;

[11] https://jitsi.github.io/handbook/docs/devops-guide/devops-guide-docker/

Levantar contenedores:

En esta configuración, Jitsi utiliza una arquitectura modular. Además del archivo base docker-compose.yml, integra servicios específicos mediante archivos YAML independientes para extender su funcionalidad:

- **Jibri (jibri.yml):** Gestiona las grabaciones y el streaming.
- **Whiteboard (whiteboard.yml):** Habilita la pizarra interactiva.
- **Etherpad (etherpad.yml):** Permite la edición de documentos compartidos en tiempo real.

Para levantar o detener el stack completo, es necesario invocar todos los archivos de configuración en una sola línea de comando para que Docker reconozca las dependencias entre ellos.

Iniciar los servicios

Ejecuta el siguiente comando para descargar las imágenes e iniciar los contenedores en segundo plano (**detached mode**):

docker compose -f docker-compose.yml -f whiteboard.yml -f etherpad.yml -f jibri.yml up -d

Detener los servicios

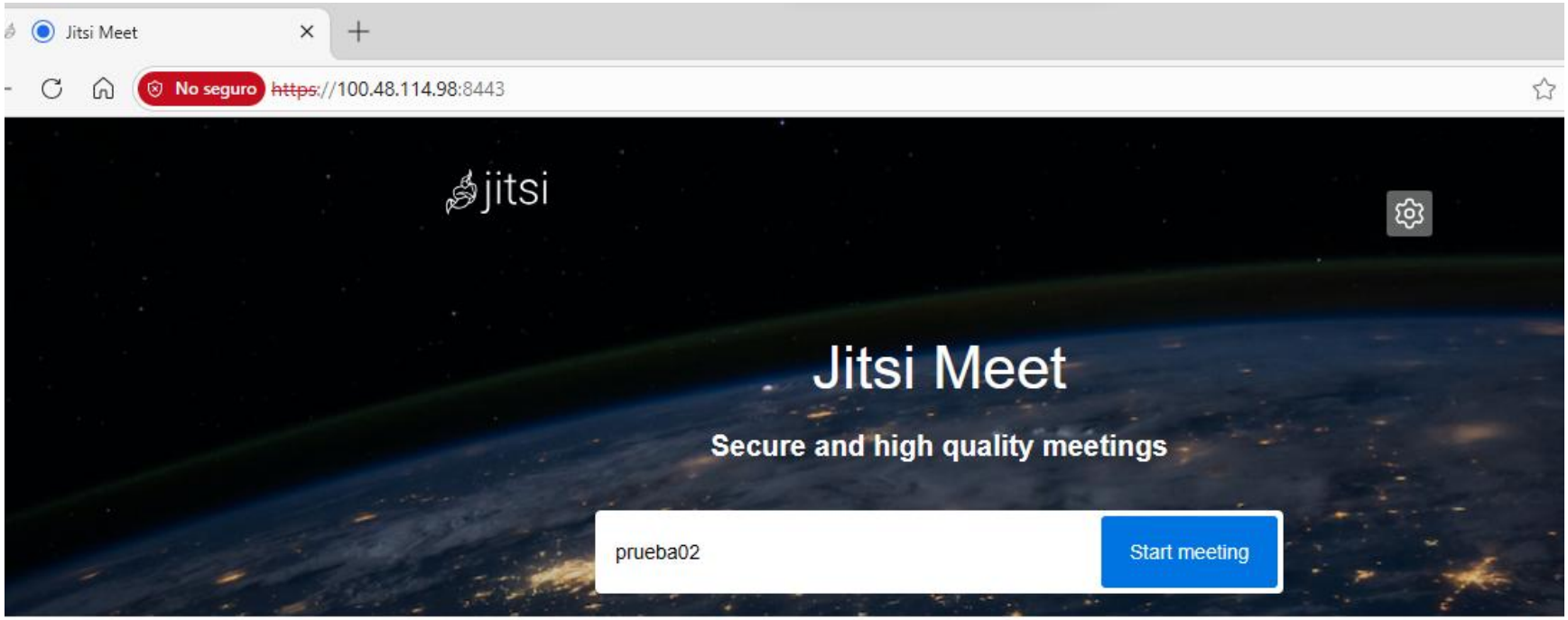
Para detener y eliminar los contenedores manteniendo la integridad de la red, utiliza:

docker compose -f docker-compose.yml -f whiteboard.yml -f etherpad.yml -f jibri.yml down

Usando Jitsi

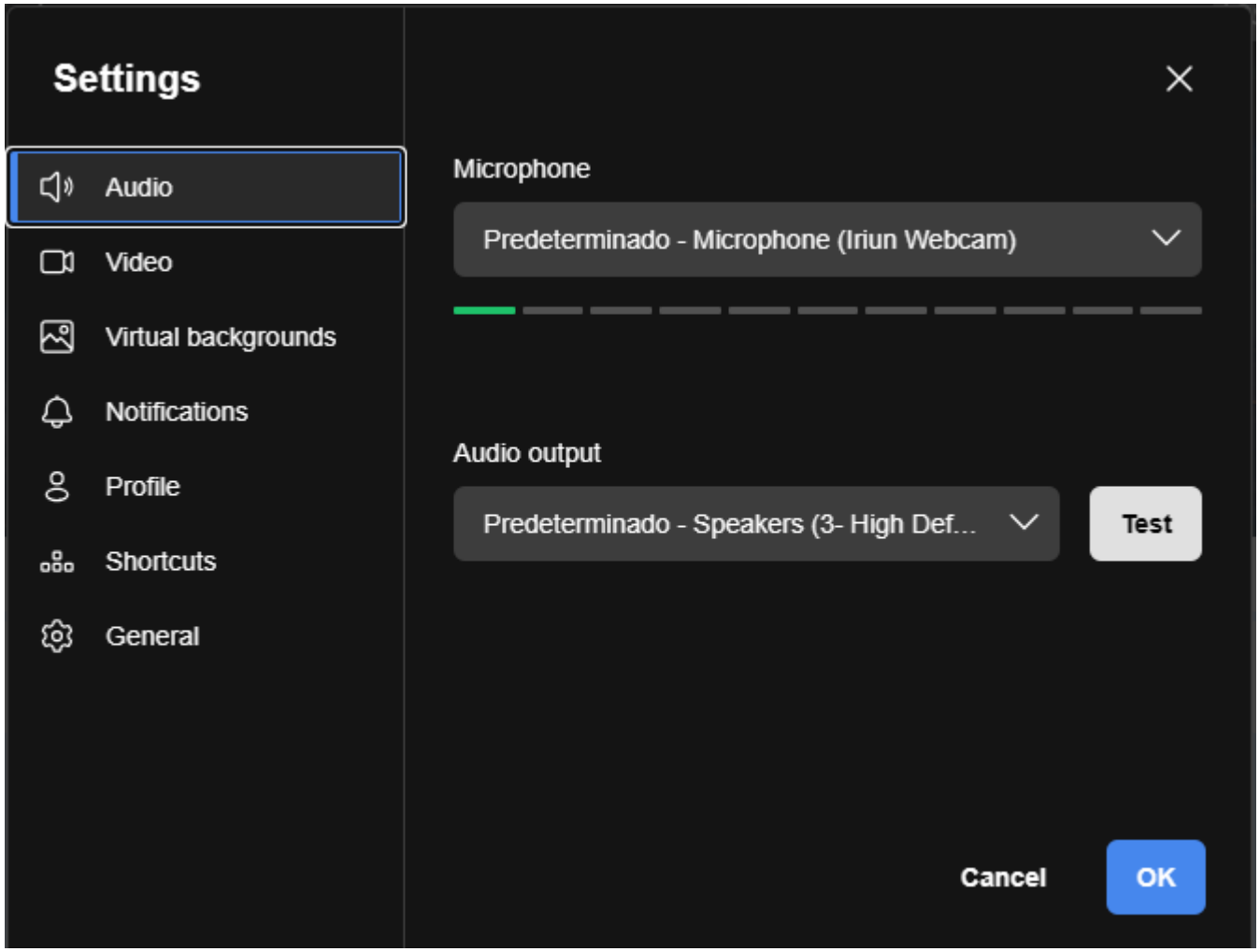
https://100.48.114.98:8443/

Ventana principal.

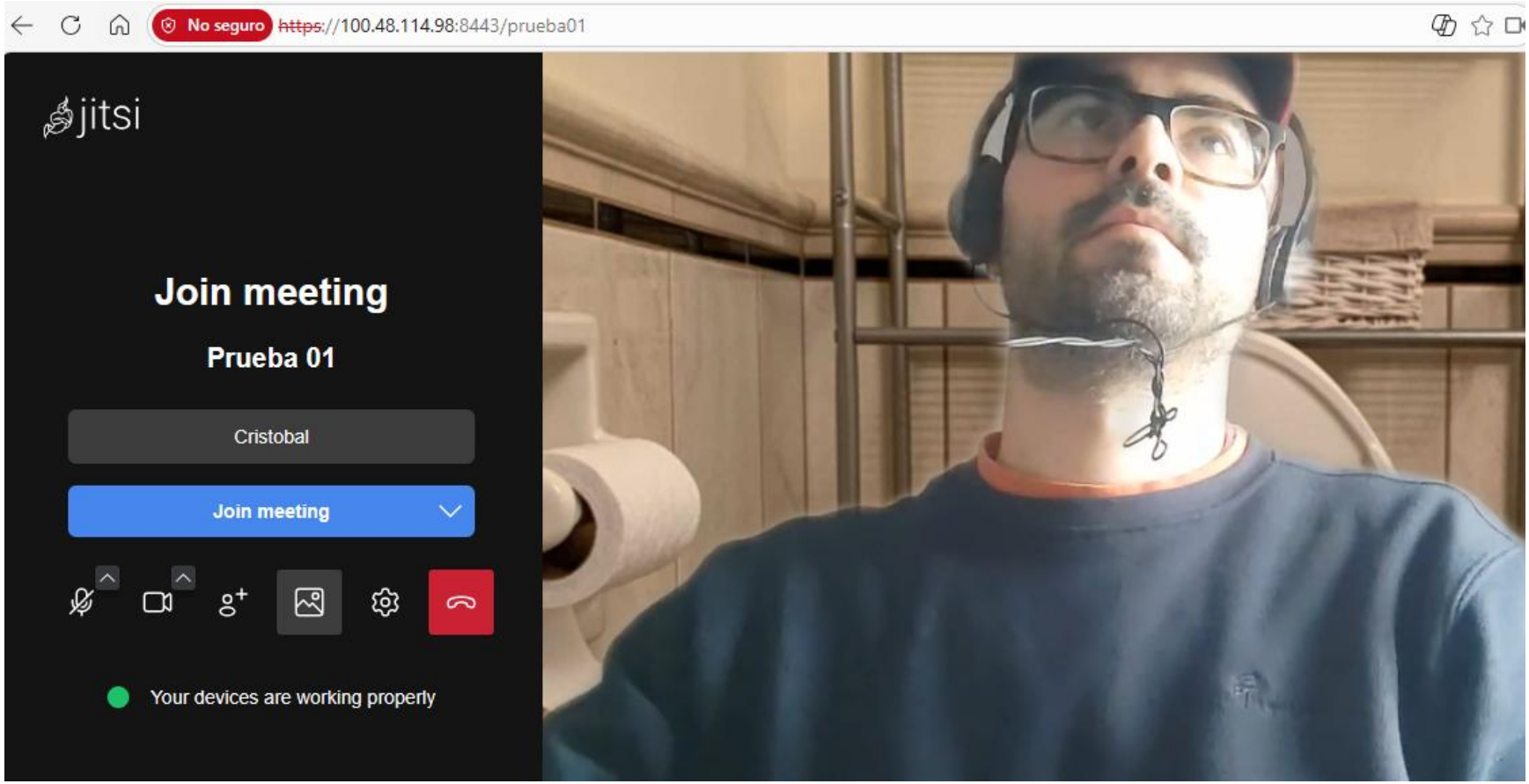




Sección de configuración:

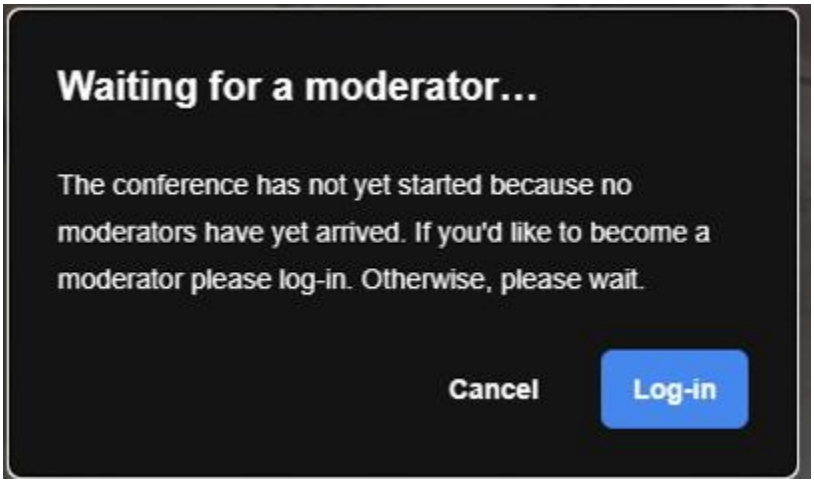


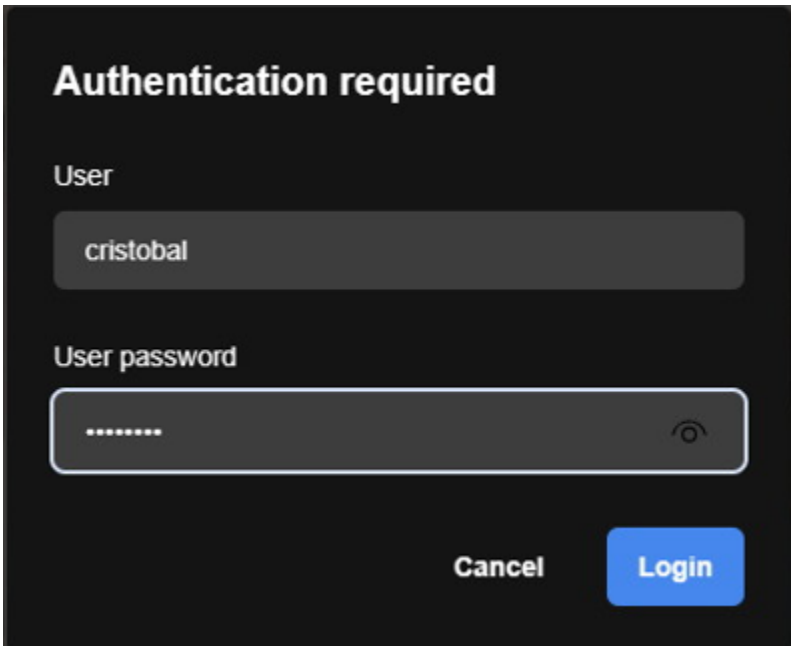
Ventana antes de meternos en la sala:



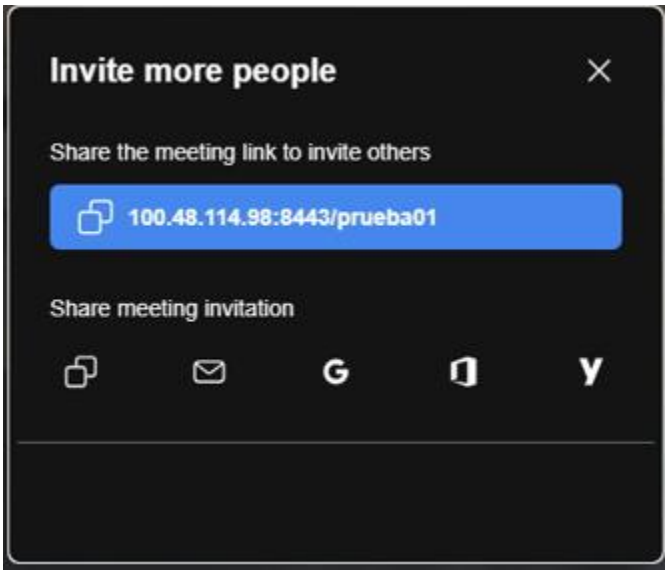
Podemos activar y desactivar cámara y micrófono, invitar a gente, elegir fondo y más.

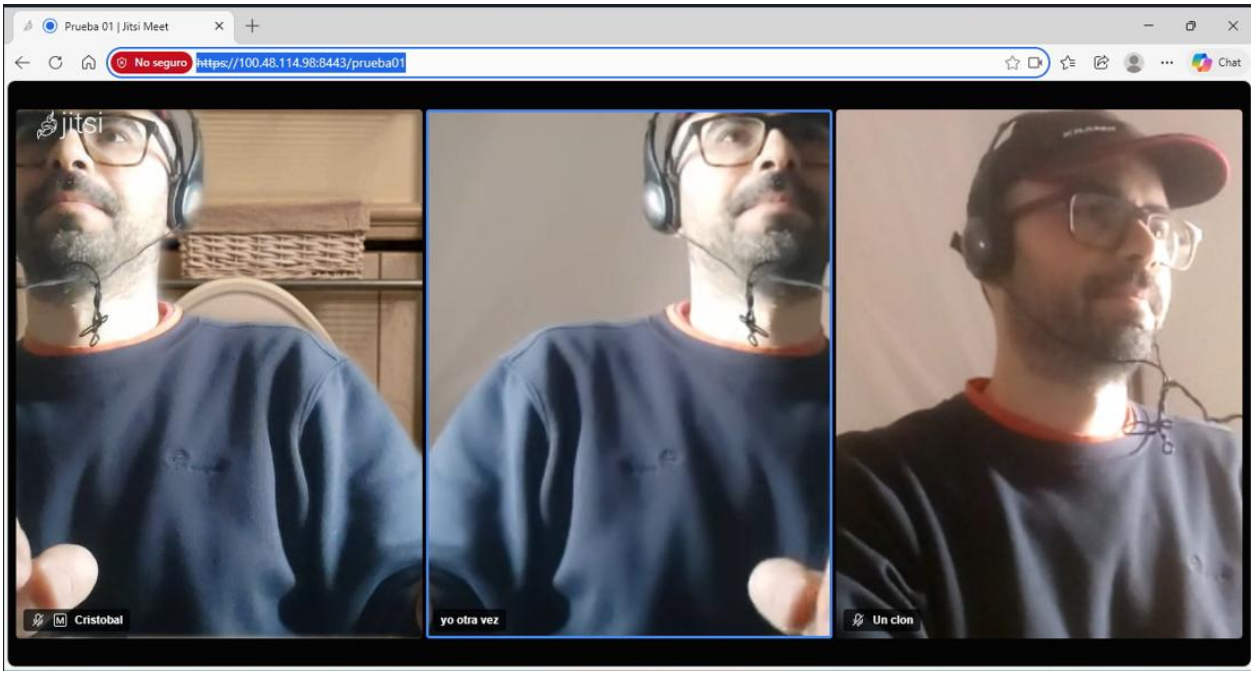
Como activos la autenticación, ahora es necesario tener una cuenta para crear e iniciar sesiones.





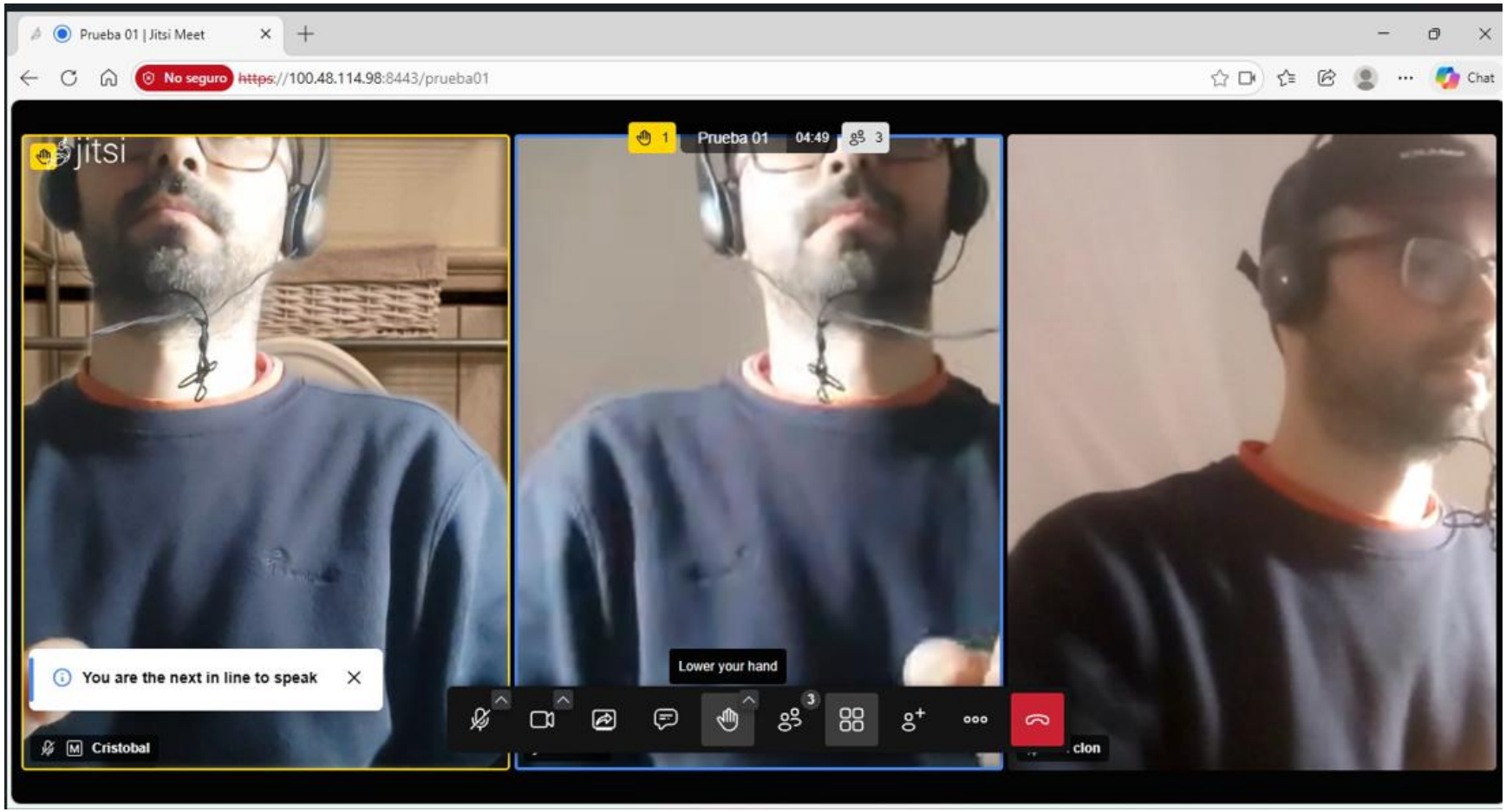
Una vez que estemos dentro podremos invitar a más gente.



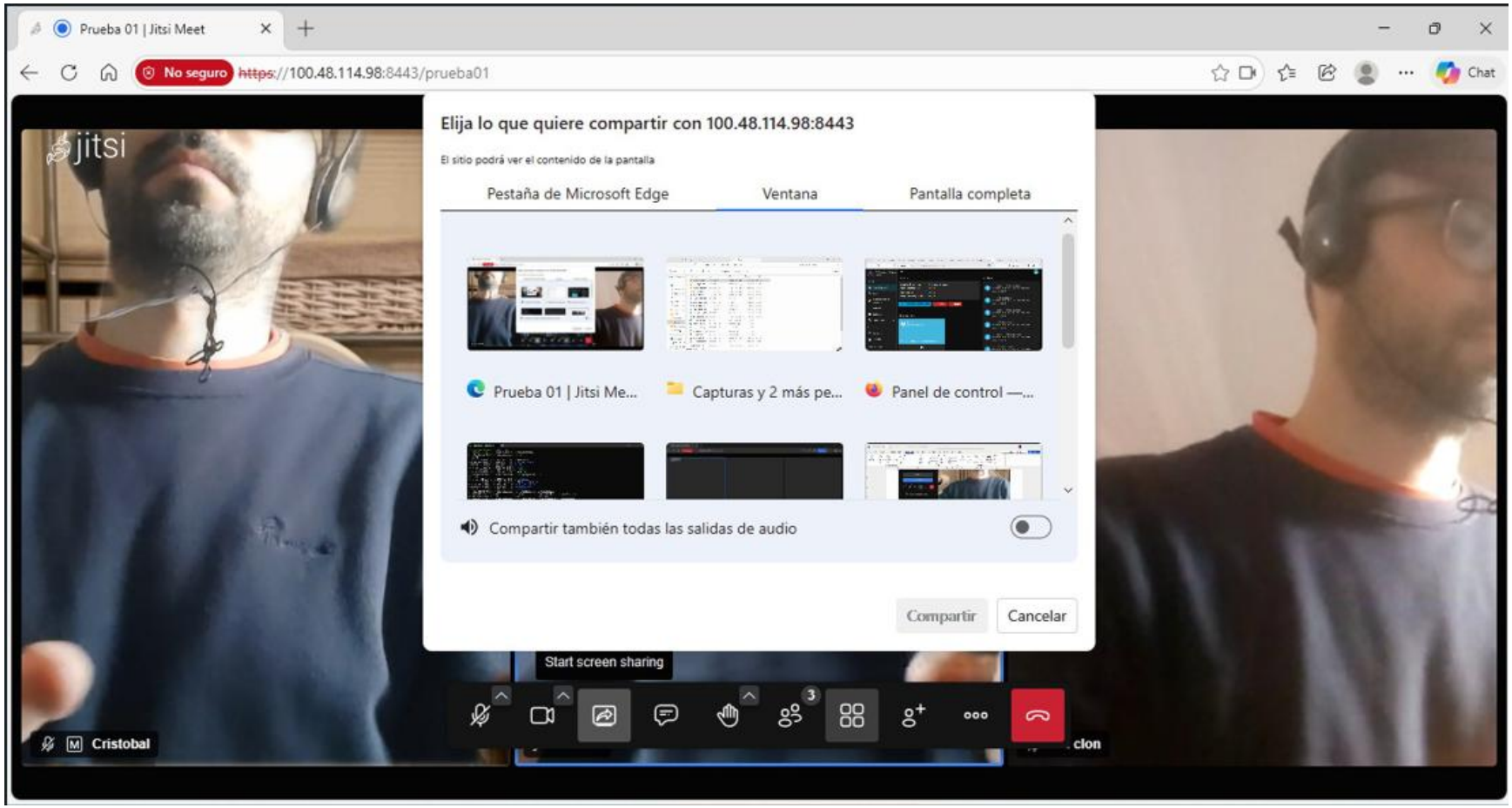


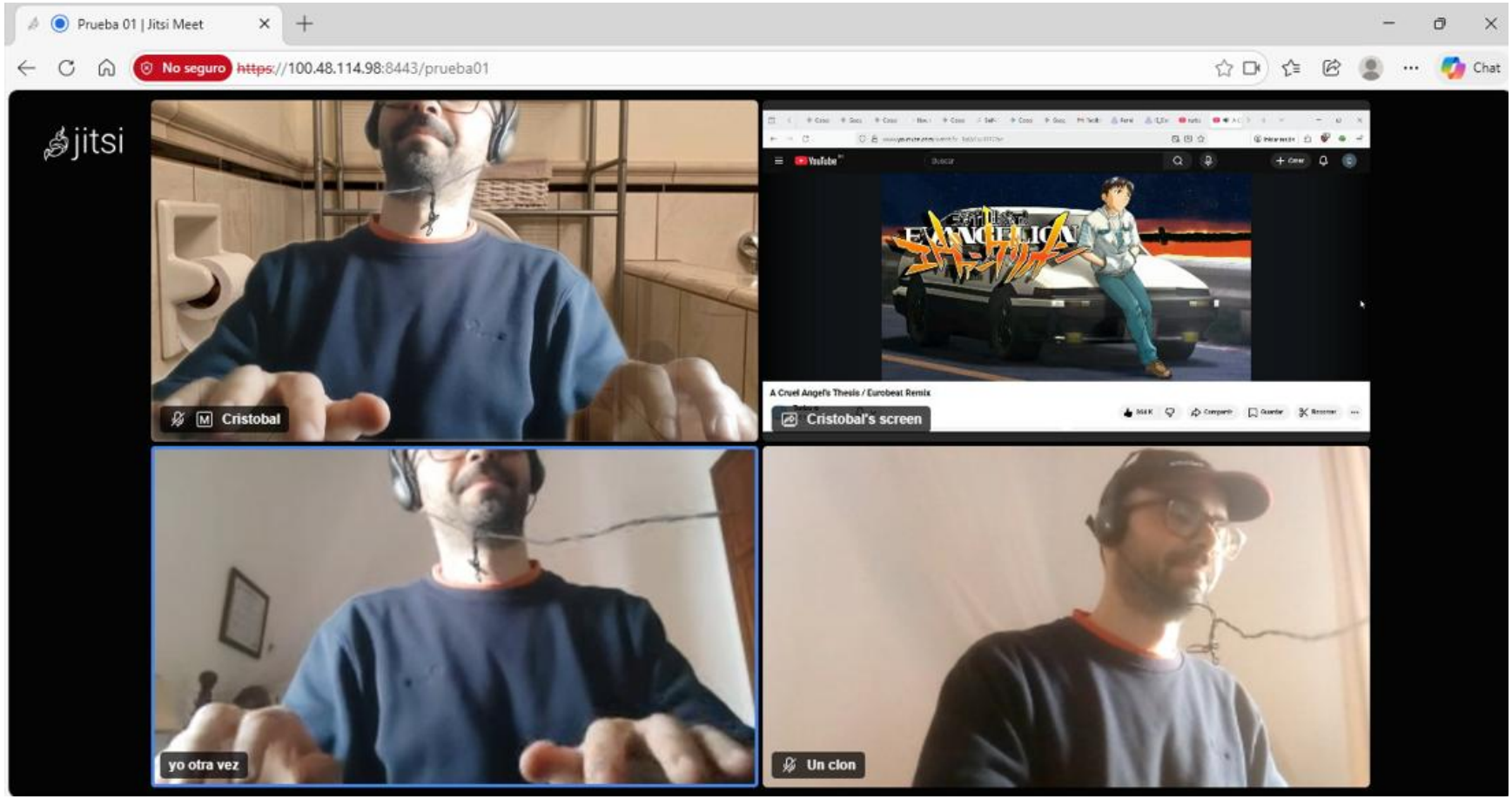
Funciones

- Podemos pedir turno de palabra:

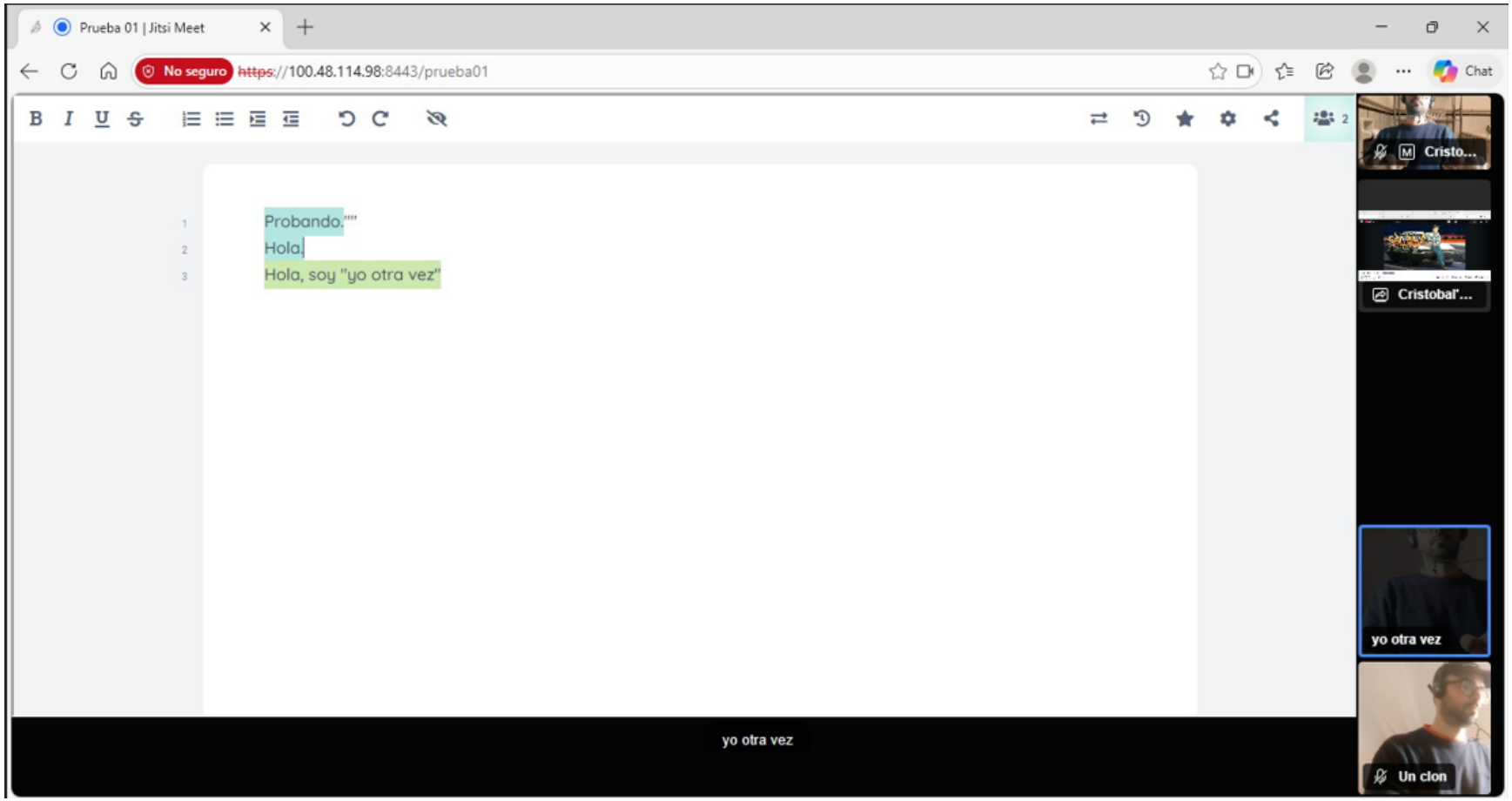


- Compartir nuestra pantalla:

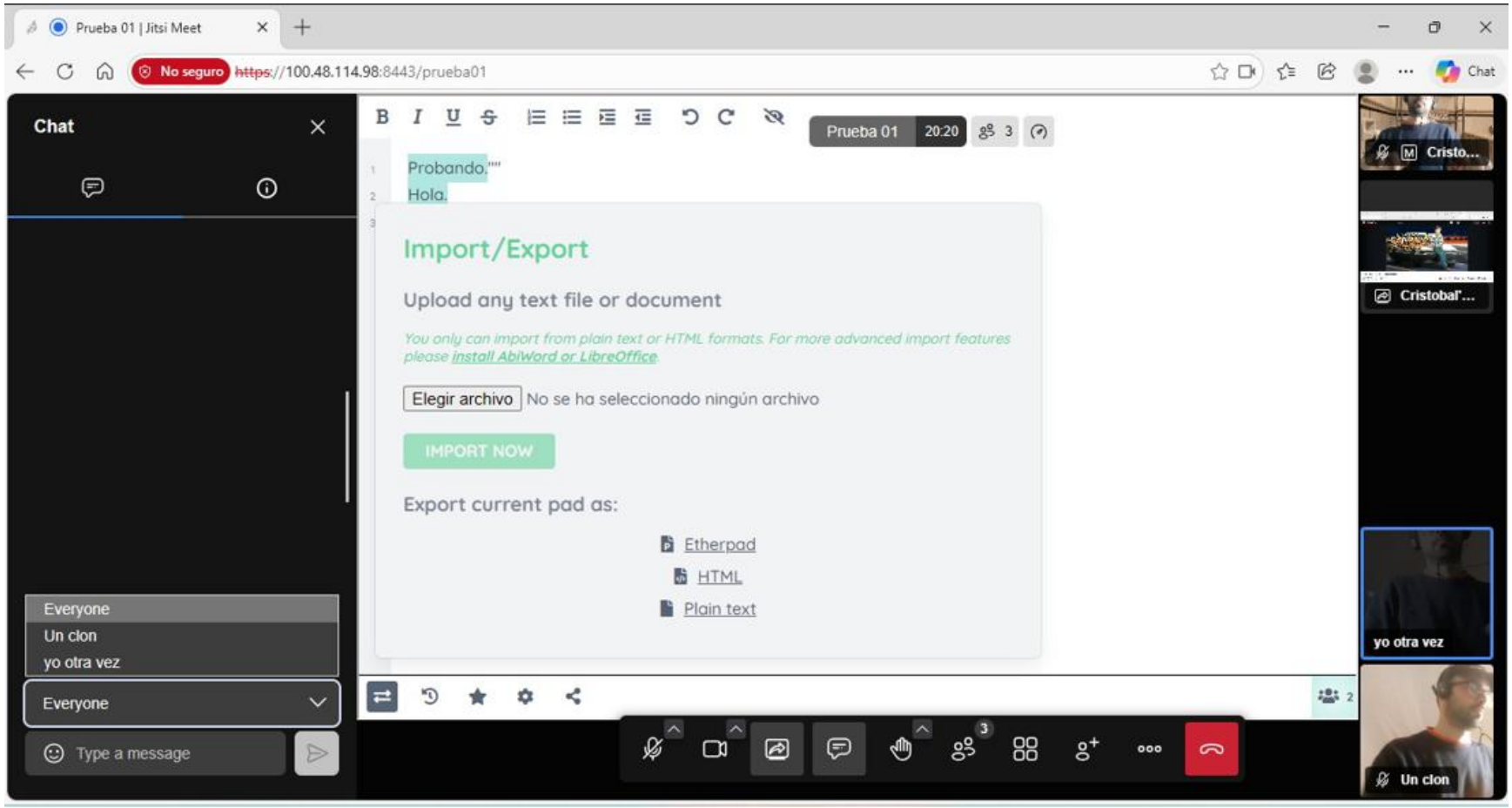



Prueba 01 | Jitsi Meet
No seguro
https://100.48.114.98:8443/prueba01
Chat
jitsi
Cristobal
A Cruel Angel's Thesis / Eurobeat Remix
Cristobal's screen
yo otra vez
Un clon

- Documento y pizarra compartida:

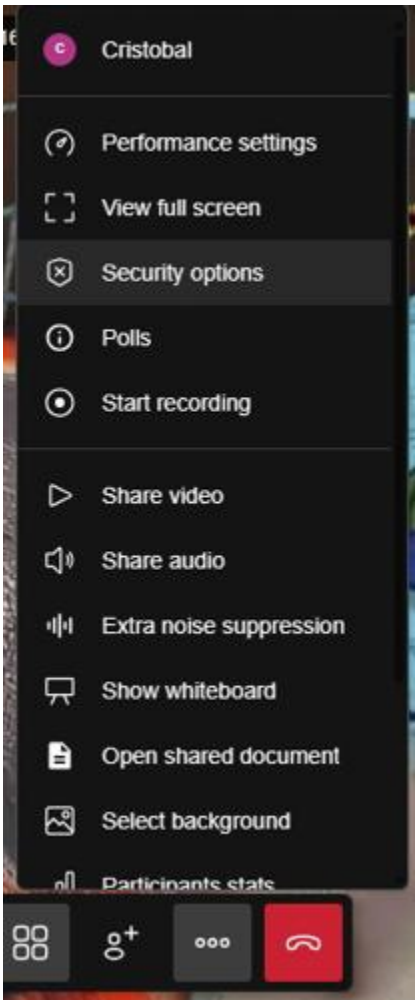


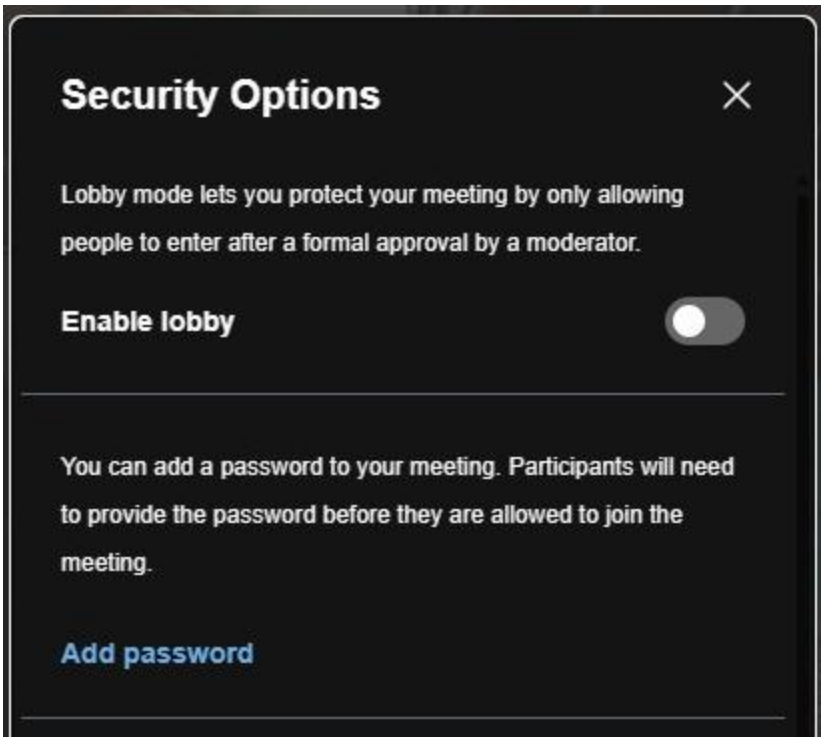
- Chat:


Prueba 01 | Jitsi Meet
No seguro https://100.48.114.98:8443/prueba01
Chat
Prueba 01
20:20
Probando.""
Hola.
Import/Export
Upload any text file or document
You only can import from plain text or HTML formats. For more advanced import features please install AbiWord or LibreOffice.
Elegir archivo
No se ha seleccionado ningún archivo
IMPORT NOW
Export current pad as:
Etherpad
HTML
Plain text
Everyone
Un clon
yo otra vez
Everyone
Type a message
Cristo...
Cristobal'...
yo otra vez
Un clon

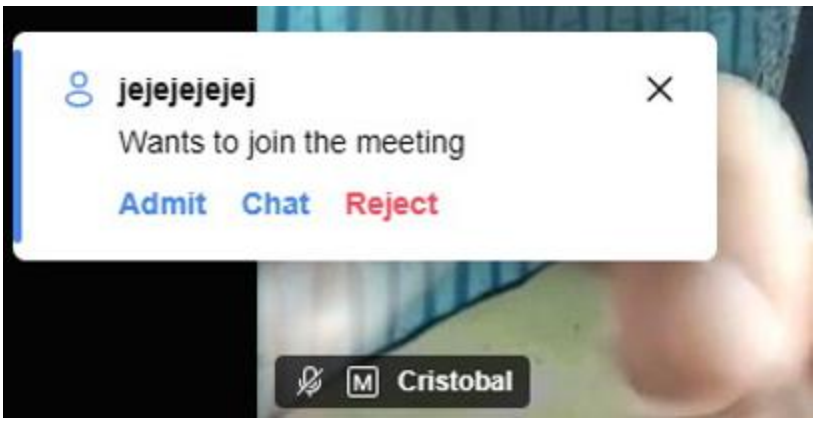
Seguridad:

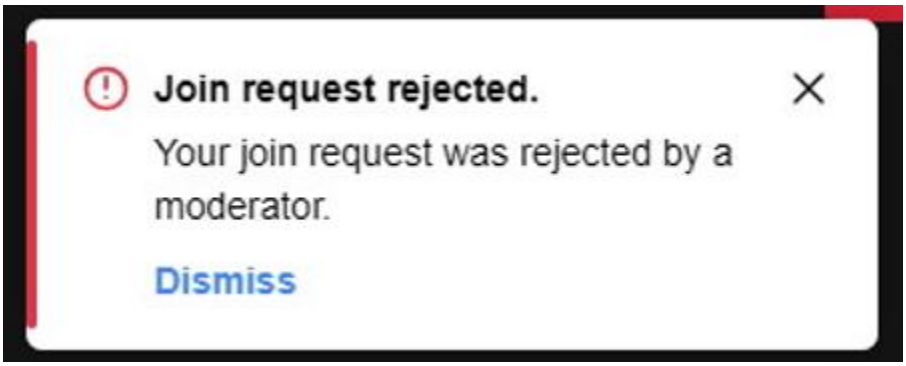
Por desgracia, las opciones de seguridad no se configuran cuando se crea la reunión, si no una vez que el moderador está dentro. Además, desaparecen una vez que todos los participantes de una reunión la abandonan.





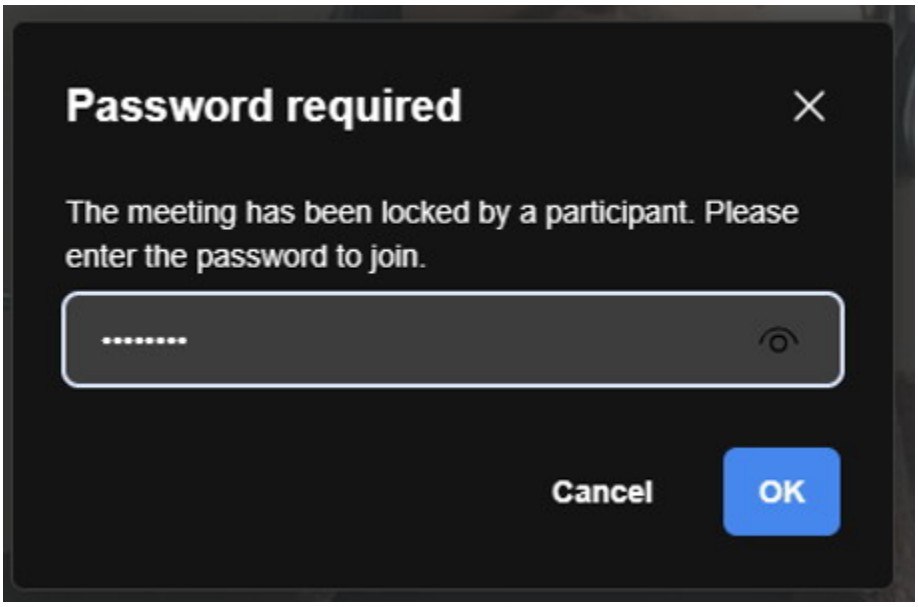
Podemos habilitar un lobby: esto obligará a realizar una petición antes de entrar en la reunión. Solo podremos entrar si nos da permiso un moderador.



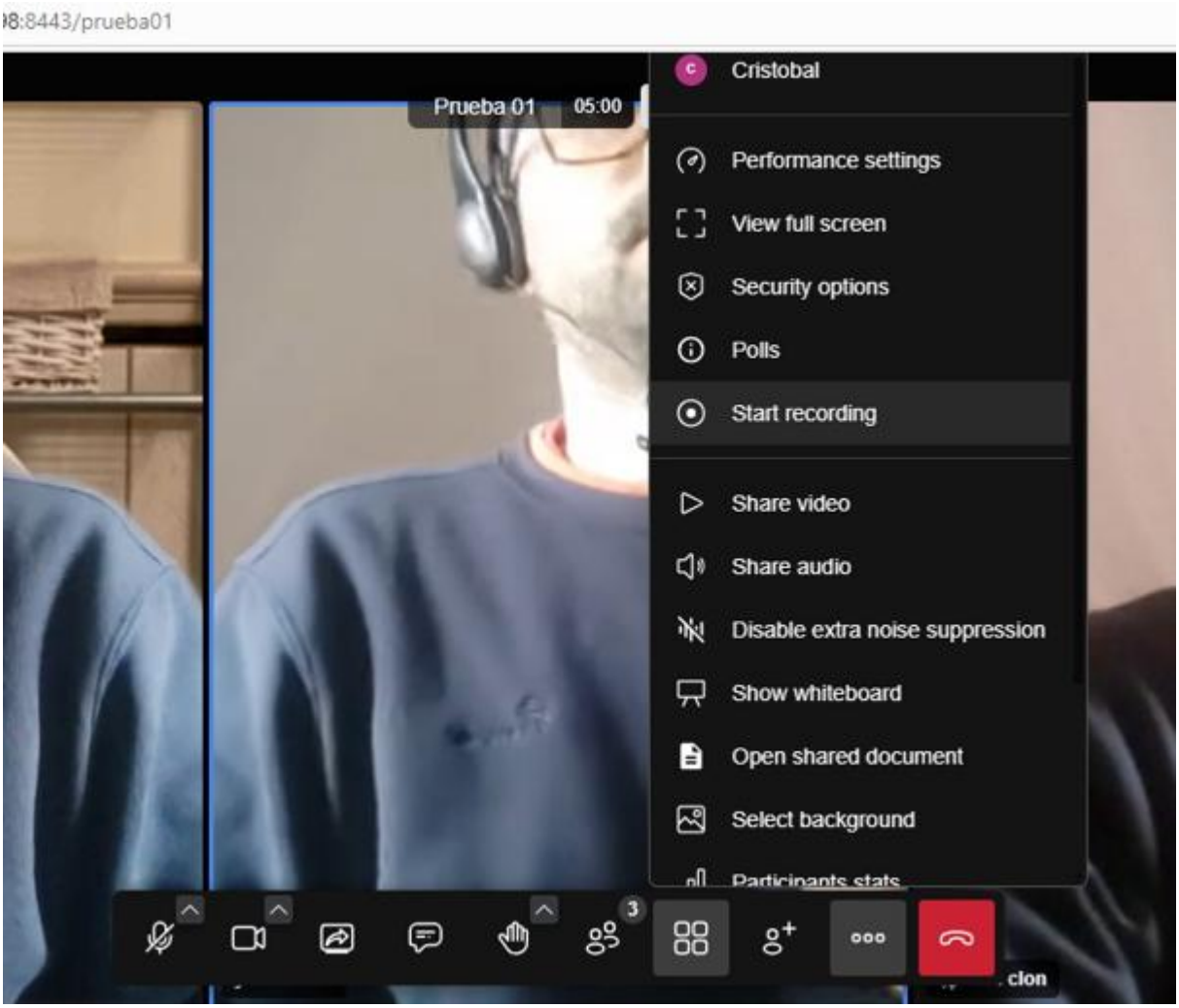


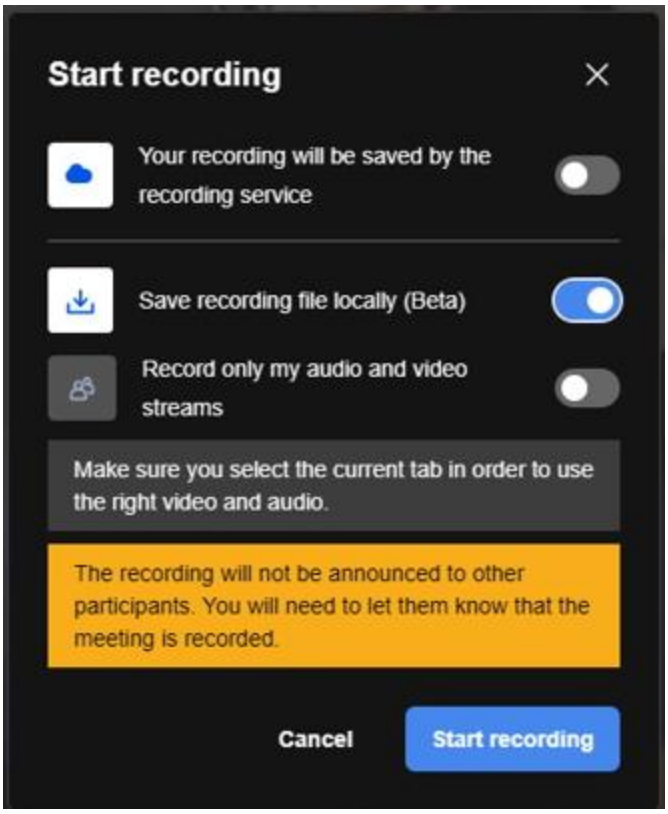
O podemos establecer una contraseña para poder acceder a la reunión.





Grabación de Videoconferencias:

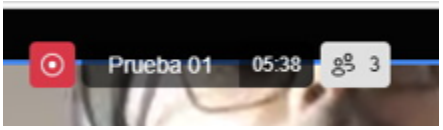




Almacenado en el servidor

Almacenado en el cliente

Cuando hacemos las grabaciones con el servidor, a todos los participantes les saldrá el símbolo de grabación.



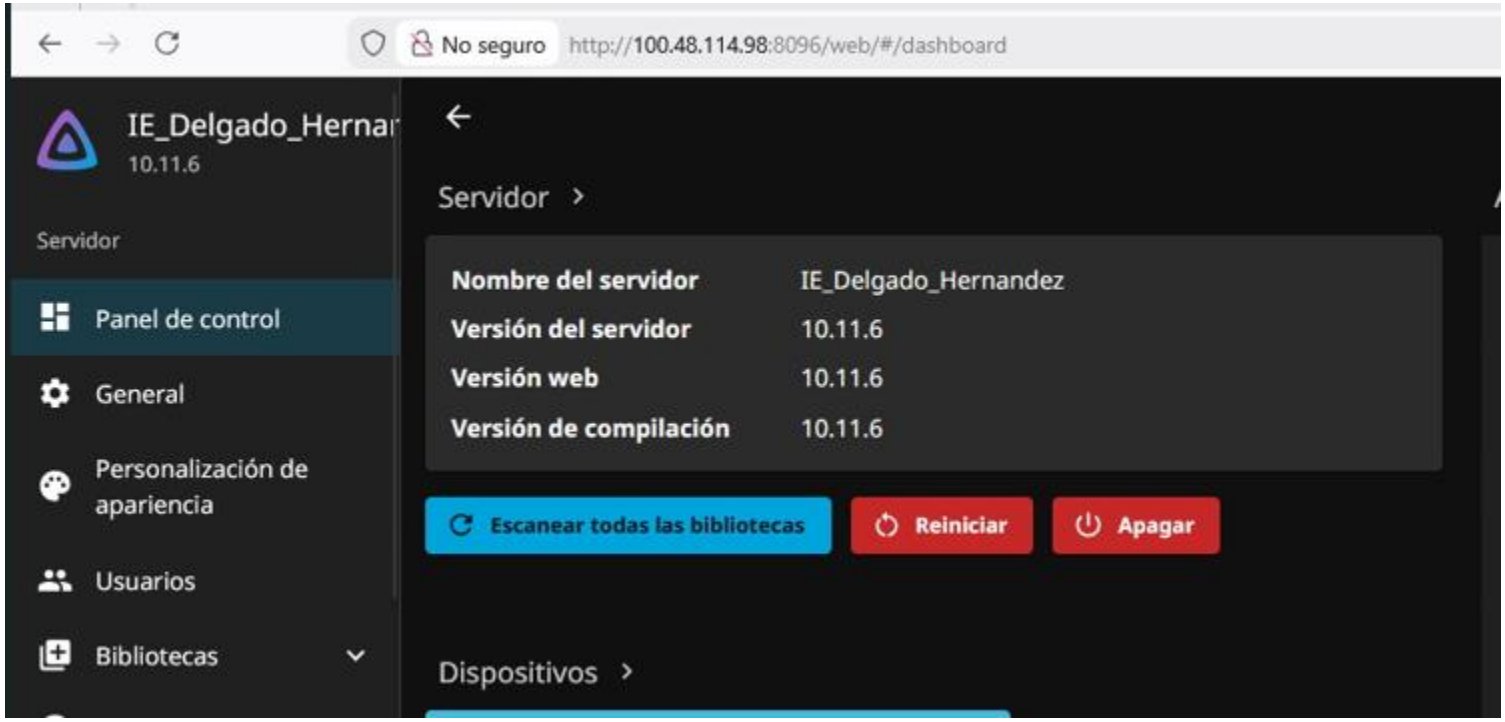
Los usuarios **No Moderadores** (Invitados) de la reunión no podrán hacer grabaciones en el servidor. Pero todos los participantes podrán hacer grabaciones en sus equipos.

Un moderador que active las grabaciones en el servidor no podrá grabar en su equipo a la vez.

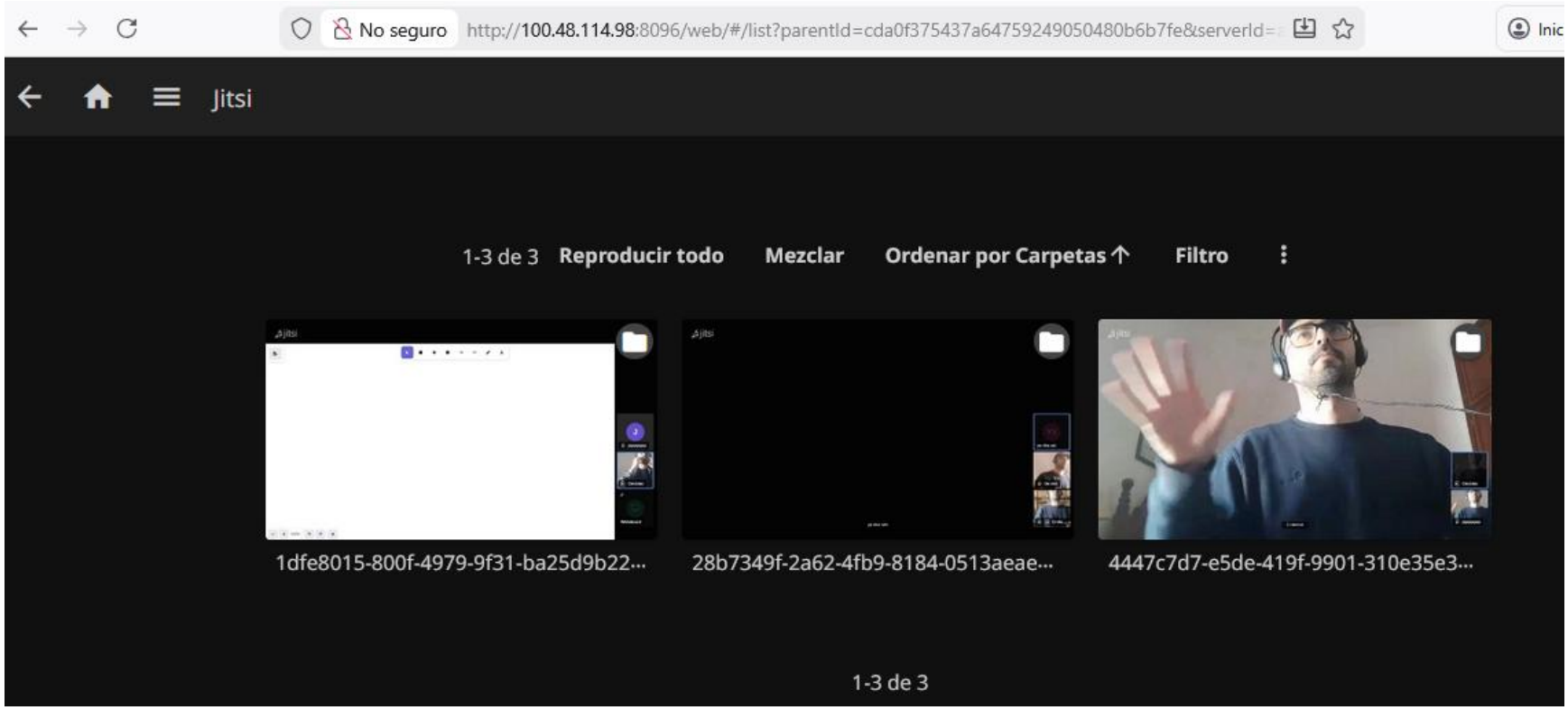
Como previamente habíamos mapeado el directorio de las grabaciones con el de **Jellyfin**, ahora podremos escanear la biblioteca para poder ver desde la biblioteca multimedia las videoconferencias grabadas en el servidor.

```
root@ip-172-31-42-182:/video_streaming# ls -l jellyfin_owncast/biblioteca/videoconferencias/
total 12
drwxr-xr-x 2 dnsmasq crontab 4096 Mar  2 11:41 1dfe8015-800f-4979-9f31-ba25d9b22cf8
drwxr-xr-x 2 dnsmasq crontab 4096 Mar  3 09:17 28b7349f-2a62-4fb9-8184-0513aeae22bd
drwxr-xr-x 2 dnsmasq crontab 4096 Mar  2 11:59 4447c7d7-e5de-419f-9901-310e35e39267
root@ip-172-31-42-182:/video_streaming#
```

Ahora vamos a Jellyfin → Panel de control → Escanear todas las bibliotecas



Y nos saldrán en la biblioteca que hayamos configurado para las videoconferencias:



Tercer bloque: Nginx Proxy Manager

```
services:
 npm:
  image: 'jc21/nginx-proxy-manager:latest'
  container_name: nginx-proxy-manager
  restart: unless-stopped
  ports:
   - '80:80'   # Tráfico HTTP (para redirección y Let's Encrypt)
   - '443:443' # Tráfico HTTPS (Tu entrada principal)
   - '81:81'   # Panel de administración (lo usamos para acceder al contenedor)
  volumes:
   - ./data:/data
   - ./letsencrypt:/etc/letsencrypt
  networks:
   - nginx_network

networks:
 nginx_network:
  external: true
```

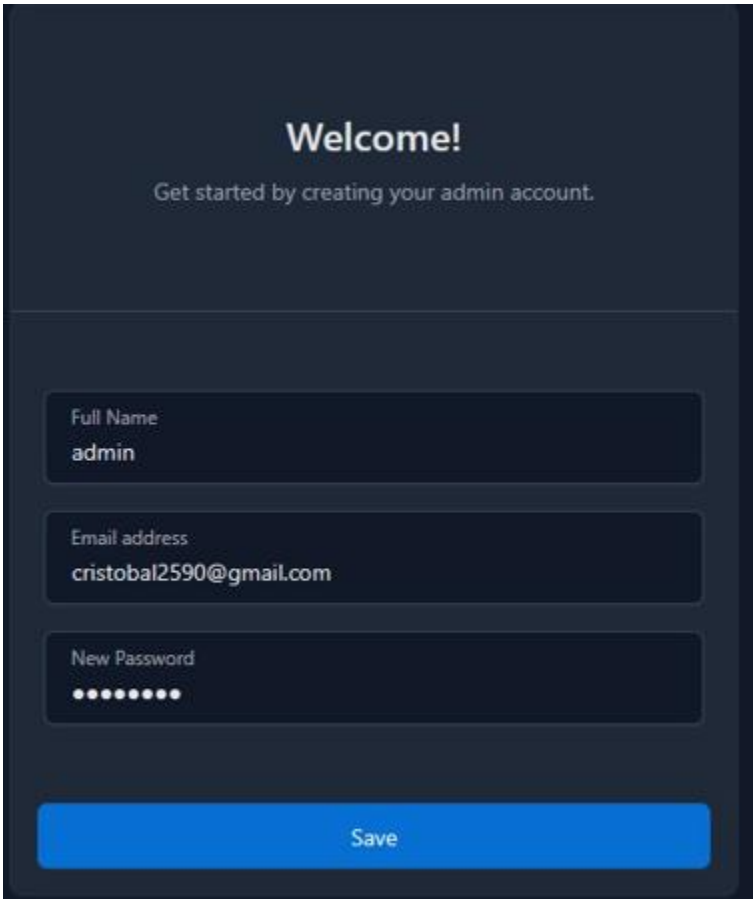
Nginx Proxy Manager (NPM) se encarga de recibir las solicitudes en tu servidor y redirigirlas a la aplicación correspondiente (evita peleas por los puertos) y también ser encarga de gestionar las credenciales de los dominios y subdominios. NPM se comunica con los servidores de **Let's Encrypt**, valida que tú eres el dueño del dominio, descarga el certificado, lo instala y, lo más importante, **lo renueva automáticamente antes de que caduque**.

NPM se apropia de los puertos estándar de Internet. Así, si alguien pone https://media.im07.institutodh.net en el navegador, el navegador contacta con el puerto 443 de tu VM, que es propiedad de NPM. Este pasará la petición al contenedor (en este caso) de Jellyfin.

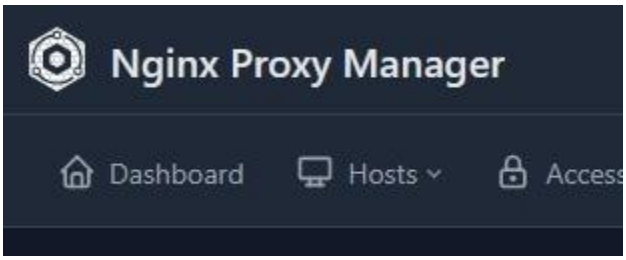
volumes: ./letsencrypt:/etc/letsencrypt: Aquí es donde NPM guarda "sus llaves". Si borras el contenedor y lo vuelves a levantar, mientras mantengas esta carpeta, **no pierdes tus certificados SSL**.

networks: nginx_network (external: true): Como explicamos antes, esto es lo que le permite a NPM "ver" a los otros contenedores (Jellyfin, Owncast, etc.) aunque estén en carpetas diferentes. NPM se conecta a la misma red "externa" que ellos y se comunican por el nombre del contenedor.

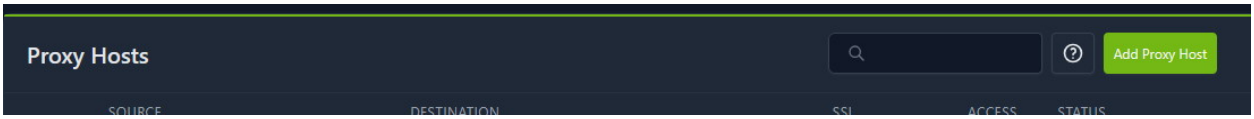
Entramos con http://100.48.114.98:81/ Tendremos que establecer un usuario administrador.



Una vez dentro, creamos un nuevo Host.

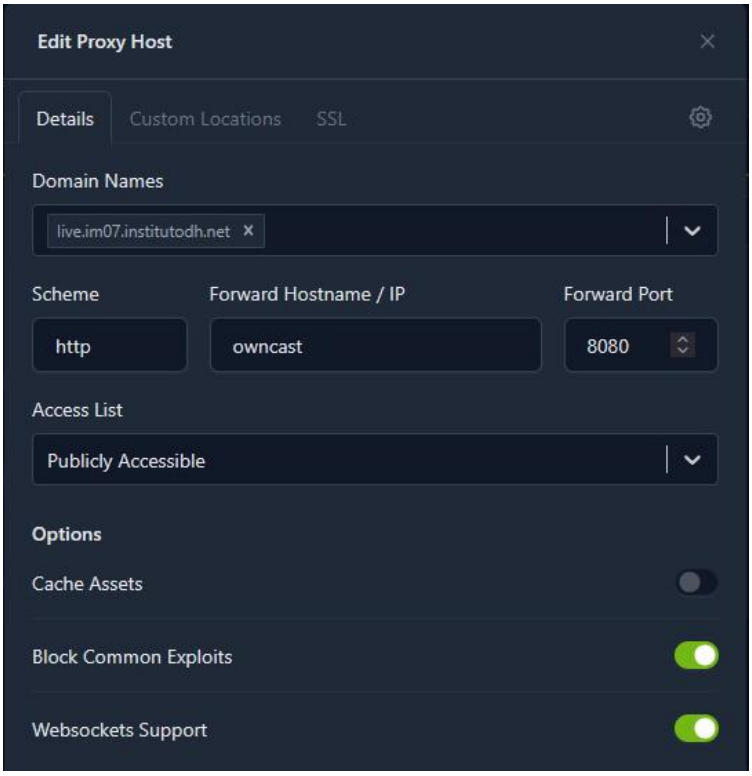


Añadir Proxy Hosts:



En nuestro caso, debemos crear uno para cada uno de los servicios (contenedores) que van a salir a Internet: **Jellyfin**, **Owncast** y el contenedor web de **Jitsi**.

Usamos esquema “http”, el nombre o ip (al estar todos dentro de la misma red en Docker, se pueden usar los nombres de los contenedores) y el puerto interno. En este punto cuando se establece la comunicación interna mediante la cual NPM interactuará con los servicios.

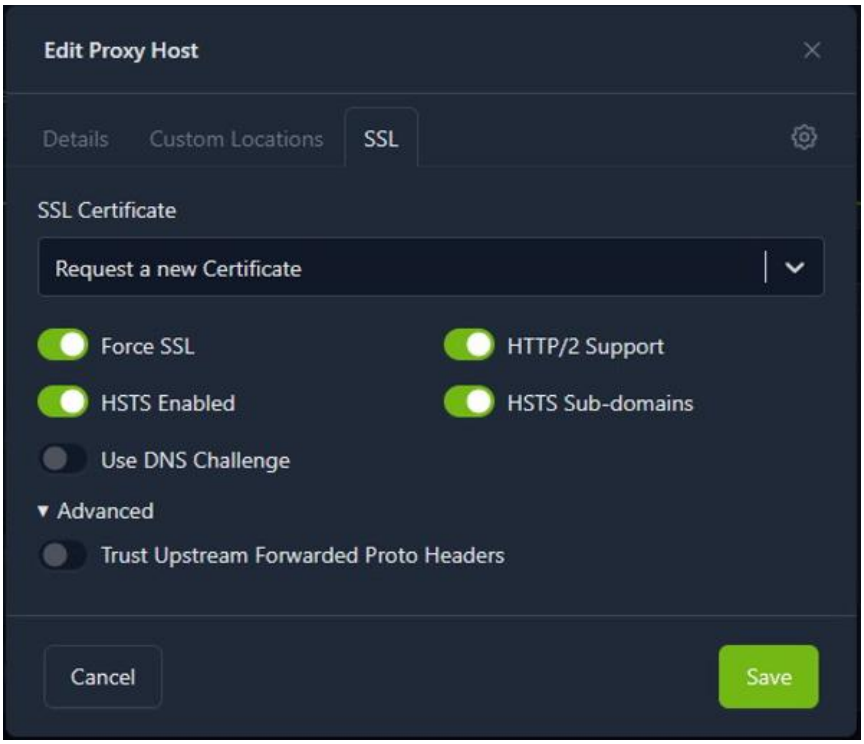


Block Common Exploits: Está activado. Esto añade una capa extra de seguridad (protección contra ataques comunes como inyecciones SQL o intentos de ejecución de código remoto), lo cual es muy recomendable al exponer servicios a internet[12].

Websockets Support: El soporte de Websockets es crítico en este tipo de servidores porque permite la comunicación bidireccional y persistente entre el cliente y el servidor, superando las limitaciones del protocolo HTTP tradicional[13].

[12] https://github.com/NginxProxyManager/nginx-proxy-manager/discussions/1290
[13] https://www.arsys.es/blog/websockets-que-son-y-como-funcionan

Ahora tenemos que hacer que gestione el certificado:



Tenemos que especificar que queremos pedir un certificado.

Force SSL: Obliga a que cualquier persona que intente entrar por http (sin seguridad) sea redirigida automáticamente a https. Esto garantiza que todo el tráfico vaya cifrado.

HTTP/2 Support: Permite el uso del protocolo HTTP/2, que es mucho más eficiente que el antiguo HTTP/1.1. Mejora la velocidad de carga de la web al permitir múltiples transferencias simultáneas sobre una sola conexión.

HSTS Enabled (HTTP Strict Transport Security): Dice a los navegadores que ***siempre*** deben conectarse a este dominio usando HTTPS. Una vez que un usuario entra la primera vez, el navegador recordará no intentar nunca más una conexión insegura.

HSTS Sub-domains: Extiende la regla anterior a todos los posibles subdominios de **im07.institutodh.net**, reforzando la seguridad de toda la estructura.

Imagen final con todos los servicios configurados con Nginx Proxy Manager:



Fíjese en SSL que tienen todos los certificados de Let's Encrypt

Ahora podremos usar las URL seguras:

Owncast: https://live.im07.institutodh.net/

Jellyfin: https://media.im07.institutodh.net/

Jitsi Meet: https://meet.im07.institutodh.net/

Configuración de usuario Becario. Gestión de contenido multimedia.

Para la administración de contenido en Jellyfin existen otras soluciones como el uso de contenedores especializados[14]. En esta ocasión vamos a usar una solución más sencilla y directa. Usaremos el protocolo SFTP.

Primero preparamos al usuario becario sin acceso a la terminal (shell) por seguridad, limitándolo solo a la transferencia de archivos.

- Crear grupo de gestión

sudo addgroup sftp_users

- Crear usuario sin acceso a shell (-s /bin/false)

sudo useradd -m -g sftp_users -s /bin/false becario

- Asignar contraseña (la que usará en FileZilla)

sudo passwd becario

Configuración de la "Cárcel" (Chroot) y Permisos

La ruta del Chroot debe ser propiedad de root y no tener permisos de escritura para nadie más.

- Propietario Root para toda la ruta hasta la biblioteca

sudo chown root:root /video_streaming

sudo chown root:root /video_streaming/jellyfin_owncast

sudo chown root:root /video_streaming/jellyfin_owncast/biblioteca

- Permisos restrictivos (755: lectura y ejecución, pero solo root escribe)

sudo chmod 755 /video_streaming

sudo chmod 755 /video_streaming/jellyfin_owncast

sudo chmod 755 /video_streaming/jellyfin_owncast/biblioteca

[14] https://www.tinymediamanager.org/

- Carpetas de contenido: Aquí sí permitimos que el becario trabaje

sudo chown -R becario:sftp_users /video_streaming/jellyfin_owncast/biblioteca/*

sudo chmod -R 775 /video_streaming/jellyfin_owncast/biblioteca/*

Configuración del Servidor SSH

Modificamos /etc/ssh/sshd_config para habilitar las contraseñas y definir las reglas del becario.

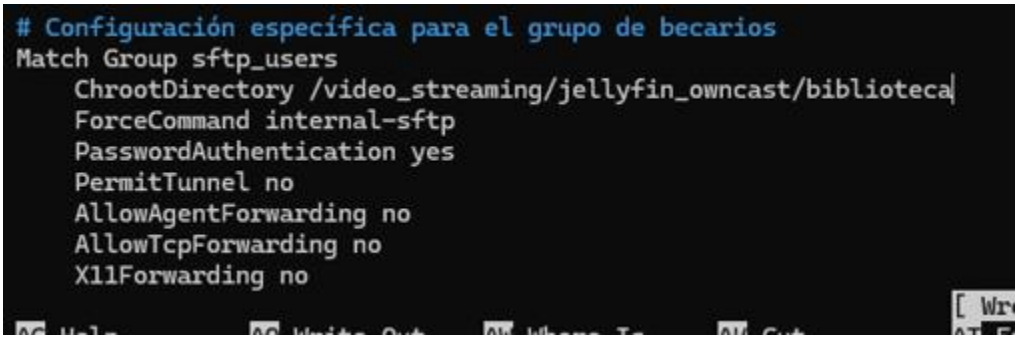
Asegúrate de que estas líneas estén activas:

PasswordAuthentication yes

KbdInteractiveAuthentication yes (o ChallengeResponseAuthentication yes en versiones antiguas)

Añade al final del archivo el bloque de restricción:

Match User becario
ChrootDirectory /video_streaming/jellyfin_owncast/biblioteca
ForceCommand internal-sftp
PasswordAuthentication yes
PermitTunnel no
AllowAgentForwarding no
AllowTcpForwarding no



Limpieza de bloqueos de AWS

En AWS Ubuntu, debemos asegurar que no haya archivos en .d anulando nuestra configuración:

- Si existe este archivo, asegúrate de que diga PasswordAuthentication yes

sudo nano /etc/ssh/sshd_config.d/60-cloudimg-settings.conf

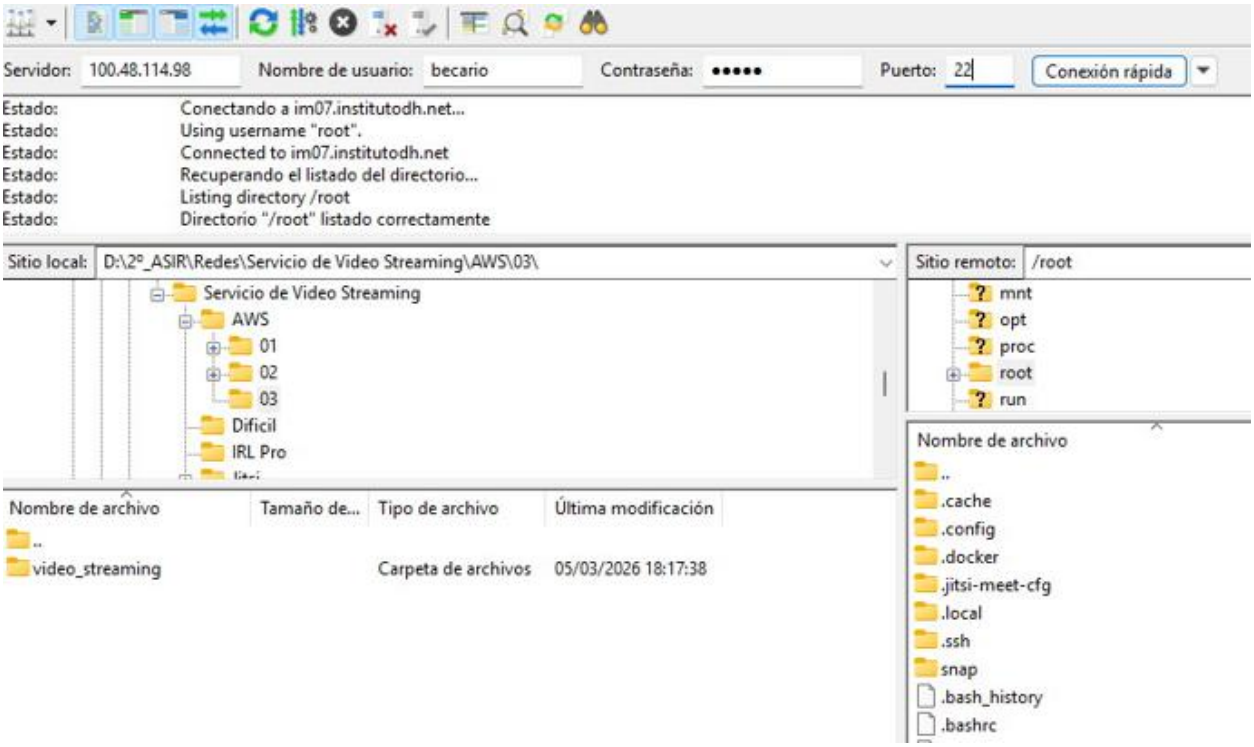
Reinicio y Aplicación

Para que el sistema reconozca los nuevos permisos y la configuración:

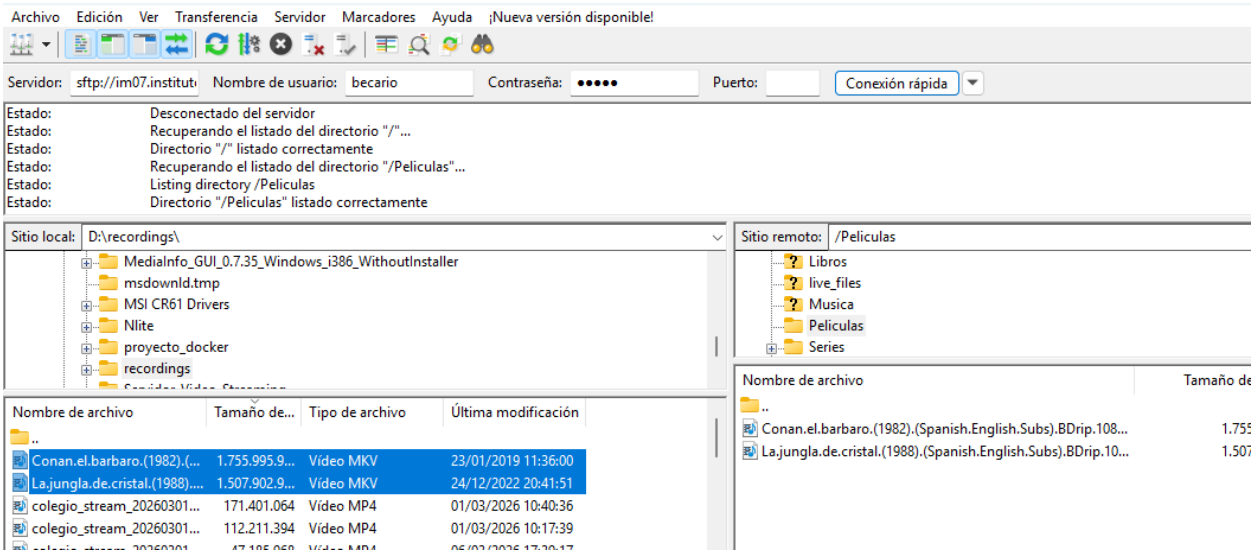
sudo systemctl daemon-reload

sudo systemctl restart ssh

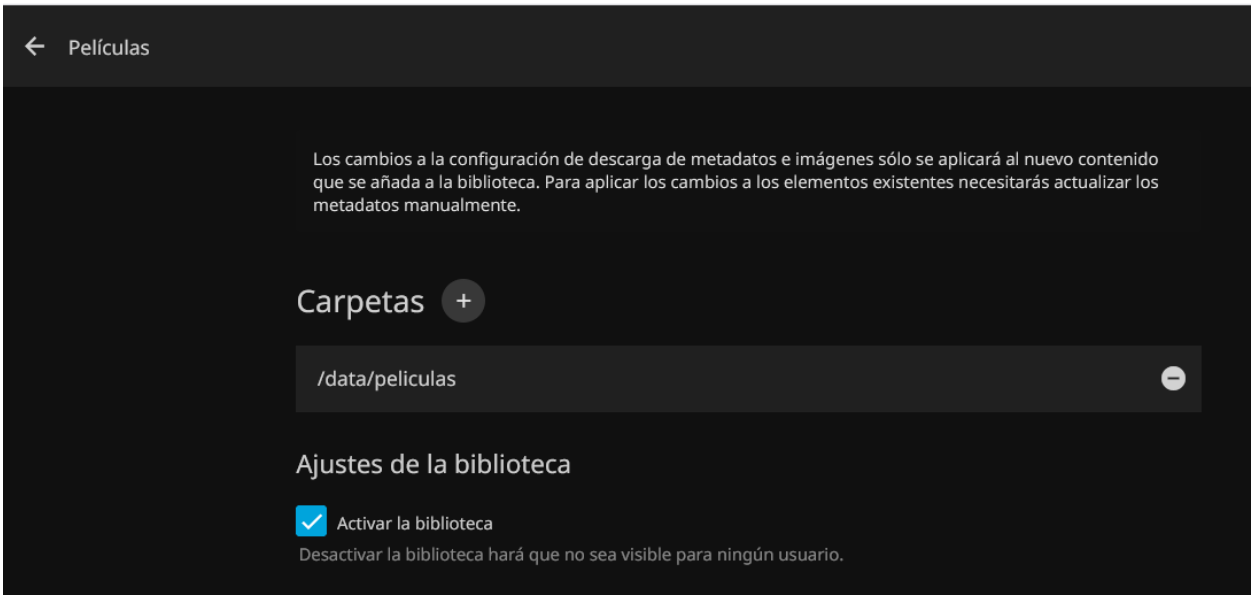
sudo systemctl restart ssh.socket



Introducimos algunas películas:

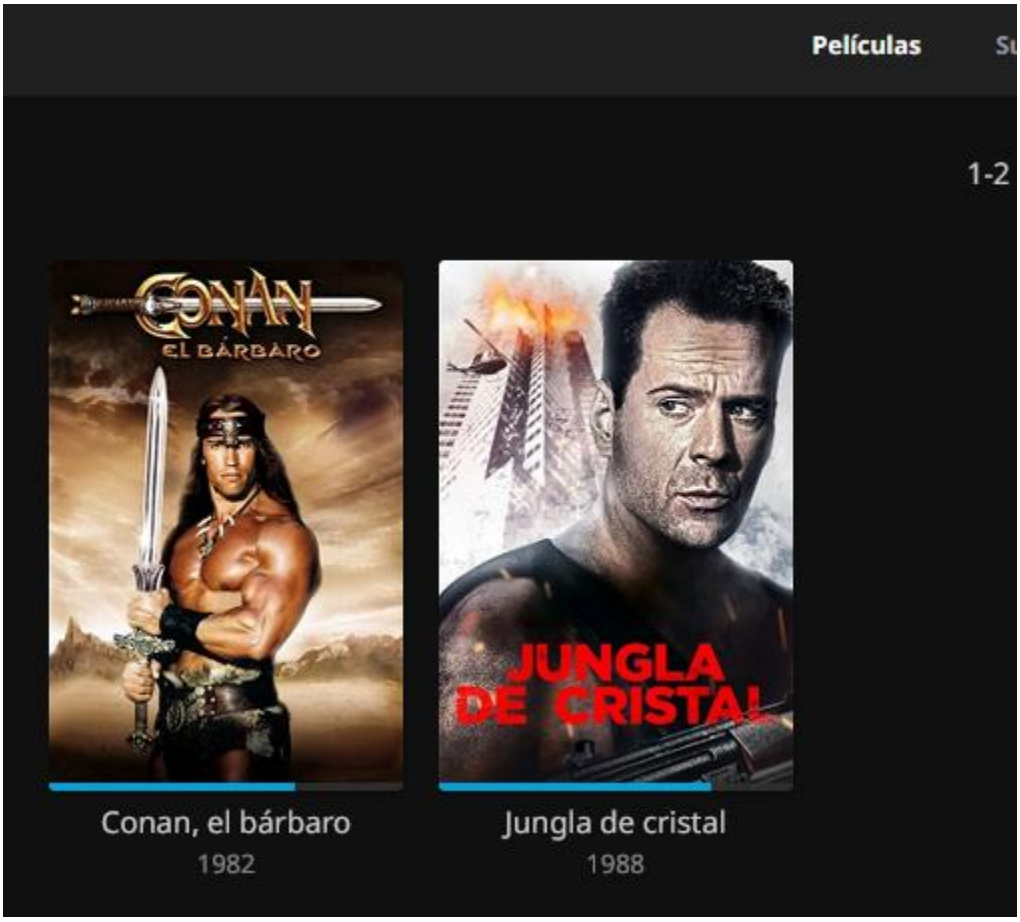


Creamos la Biblioteca “Películas”.



Y después escaneamos bibliotecas en el Panel de Control.

Nos deben de salir en su correspondiente sección. Si está configurado, se bajará automáticamente los metadatos y carátulas



Scripts para la gestión de contenedores.

Como el proyecto ha ido avanzando por bloques y algunos servidores vienen con su propio archivo "**docker-compose.yml**", he decidido que en vez de unirlos todos en uno solo, es mejor mantenerlos cada uno en su sitio. Para facilitar las tareas de levantar, parar y recrear, se hace uso de scripts.

"down_all.sh"

"rebuild.sh"

"up_all.sh"

A cada uno se le ha dado permisos de ejecución:

chmod +x "script"